A BEGINNER-TO-INTERMEDIATE COURSE

musrSim

A Hands-On Geant4 Tutorial

Simulating muon transport, detection and decay — from a text macro to a physics result.

TDLI Muon Physics Group

Tsung-Dao Lee Institute · Shanghai Jiao Tong University

Second edition, October 2026 · for `musrSim-upgrade` · Geant4 11 + ROOT 6 + Python

About this edition

This tutorial teaches Geant4 simulation through `musrSim`, using the command syntax, ROOT output and build system of the `musrSim-upgrade` repository (`github.com/kimsiang/musrSim-upgrade`). The commands, branch names and file names in this book were checked against that source. The two worked examples — `first_sim.mac` and the capstone `tf_musr.mac` — ship in the repository's `run/` folder, and every number quoted from them was produced by actually running them.

CHECK YOUR BUILD

`musrSim` is a living, forked codebase — PSI, SJTU and individual groups each carry local changes. The *standard Geant4* commands (`/run`, `/vis`, `/tracking`, `/control`, `/gps`) are universal, but the `/musr/...` and `/gun/...` commands are defined *in your build's source*. If a command in this book is rejected, that is your cue to check the code, not a typo to fight. Chapter 3 shows how to list what your executable understands, and Chapter 10 how to find any keyword in the source in ten seconds.

WHAT CHANGED SINCE THE FIRST EDITION

The first edition was written for the older `musrsim-sms` distribution on Geant4 10.7. This edition follows the cleaned-up `musrSim-upgrade` code:

- **Geant4 11 and modern CMake.** The code now builds against Geant4 11 (tested with 11.4.3) and ROOT 6 (tested with 6.40), with CMake ≥ 3.16. Geant4 10.7 still works. Field-accuracy limits and optical-property names that changed in Geant4 11 are handled automatically.
- **One physics list, configured from the macro.** `musrSim` always uses its own `musrPhysicsList`, and *your macro's* `/musr/command process` lines decide which electromagnetic processes exist. A macro without them simulates muons that never stop (Section 3.5.1).
- **Cosmic-ray generators built in.** EcoMug and CRY are now part of the code, driven by `/gun/ecomug/...` and `/gun/cry/...` commands (Appendix E).
- **Run modes.** The executable always needs a macro. Use `../musrSim macro.mac idle` to get an interactive prompt after the macro has run.
- **Correct batch seeding.** Only `/musr/run/randomOption 1` makes parallel jobs statistically independent (Section 8.4).
- **Output names** follow `data/musrSim_jp<NNN>[_<name>].root`, with a zero-padded run number.

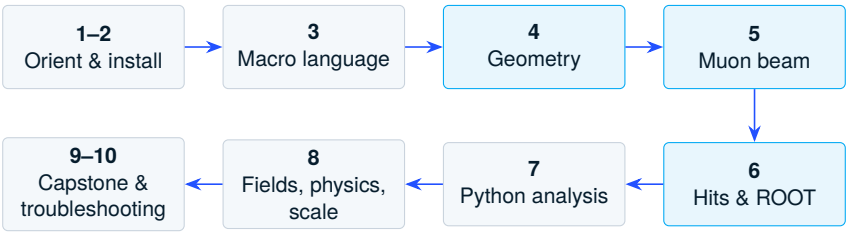
Typeset with `xelatex` in Latin Modern (body), TeX Gyre Heros (headings) and DejaVu Sans Mono (code). Source: `doc/musrSim_tutorial.tex`; build with `latexmk -xelatex`. Figures are regenerated by `doc/scripts/make_figures.py`.

How to use this book

Who this is for. Anyone who wants to run muon simulations and does not yet know Geant4. We assume you are comfortable in a Unix shell and can read a little C++ and Python. No Geant4, ROOT or Monte-Carlo experience is required.

The three things you actually need. A working `musrSim` executable (Chapter 2), a text editor, and a Python environment for analysis (Chapter 7). That is the whole toolchain.

THE LEARNING PATH



Shaded = the core craft. If you only have an afternoon, do Chapters 2, 3, 4 and 6.

HOW TO READ THE CALLOUTS

KEY POINT
The one idea on the page that, if you remember nothing else, you should remember.

TIP
A shortcut, a good habit, or a small trick that saves you time.

COMMON PITFALL
A mistake beginners actually make. Read these — they are the errors that cost an afternoon.

PHYSICS INTUITION
The physics behind the code: why the numbers come out the way they do.

UNDER THE HOOD
What Geant4 or `musrSim` is doing internally. Optional on a first read, useful when debugging.

CHECK YOUR BUILD
Something that may differ in your fork. A prompt to verify against your own source, not to trust blindly.

CONVENTIONS FOR CODE

Shell commands, macro files (`.mac`), C++ and Python are shown in monospaced blocks with a blue rule on the left and a caption underneath. A long line that does not fit is wrapped and marked with ↪; type it as *one* line. Comments begin with `#`. Paths such as `build/run` are relative to the top of the repository.

> **COMMON PITFALL**
>
> The single most important convention in this book. In every `/musr/command construct` line, box and tube dimensions are *half-lengths*, following Geant4's `G4Box` and `G4Tubs`. A "`100 100 5`" box is $200 \times 200 \times 10$ mm. Beginners routinely build detectors twice the size they intended. We flag this again in Chapter 4, but internalise it now.

Contents

About this edition i

How to use this book ii

1 The big picture: what `musrSim` is 1

1.1 Geant4, in one paragraph . . . 1

1.2 What `musrSim` adds . . . 1

1.3 The three layers . . . 2

1.4 The life of one event . . . 2

1.5 Why muons, and why this toolkit . . . 3

1.6 What you will build in this book . . . 4

2 Installation and first light 5

2.1 What you need . . . 5

2.2 The fastest path: one conda environment . . . 5

2.3 Installing the dependencies yourself . . . 6

2.3.1 Linux (Ubuntu/Debian) . . . 6

2.3.2 macOS . . . 6

2.3.3 Make the environments available . . . 6

2.4 Get and build `musrSim` . . . 7

2.5 First light: run the bundled example . . . 7

2.6 See the geometry . . . 8

2.7 Troubleshooting the build . . . 8

3 The macro language, and your first simulation 10

3.1 Three families of commands . . . 10

3.2 How `musrSim` reads a macro: twice . . . 11

3.3 List the real commands . . . 12

3.4 Comments and units . . . 12

3.5 Your first macro, from a blank file . . . 12

3.5.1 The physics block . . . 14

3.5.2 Run it . . . 14

4 Building geometry with `construct` 16

4.1 The construct grammar . . . 16
4.2 Half-lengths: the number-one beginner trap . . . 17
4.3 Mothers, daughters, and local coordinates . . . 17
4.4 Rotations . . . 18
4.5 Materials and solids . . . 19
4.6 Making a volume a detector, and the magic names . . . 20
4.7 Seeing it, and checking for overlaps . . . 21

5 The muon beam and its spin 23

5.1 Species, vertex, and spread . . . 23
5.2 Momentum and direction . . . 24
5.3 The muon spin . . . 24
5.4 Decay timing . . . 25
5.5 A realistic beam from TURTLE . . . 25
5.6 When to use the General Particle Source . . . 25
5.7 Cosmic-ray muons . . . 26

6 Hits and the ROOT output tree 27

6.1 What a hit is . . . 27
6.2 The shape of tree `t1` . . . 27
6.3 Selecting a detector with `det_ID` . . . 28
6.4 Controlling the output . . . 29

7 Analysing the tree in Python 31

7.1 Opening the file . . . 31
7.2 Energy spectra: flattening and masking . . . 32
7.3 The muon lifetime . . . 32
7.4 A reusable loader . . . 33

8 Fields, physics, and running at scale 35

8.1 Magnetic and electric fields . . . 35
8.2 The physics list, exactly . . . 36
8.3 Optical photons . . . 37

8.4 Running at scale 38

9 Capstone: a transverse-field asymmetry measurement 41

9.1 The physics in one paragraph 41

9.2 The macro 42

9.3 The analysis 43

9.4 What you should see 45

10 Troubleshooting, and reading the source 47

10.1 Common failures at a glance 48

10.2 When a command is rejected 48

10.3 A map of the source 49

10.4 Validation habits 49

A Command quick reference 51

B ROOT output branch reference 53

C Materials and solids reference 55

D An annotated skeleton macro 56

E Cosmic-ray muons: EcoMug and CRY 58

E.1 EcoMug 58

E.2 CRY 59

E.3 Legacy option 59

E.4 References 59

F Environment quick reference 61

G Glossary 63

H Further reading 65

Chapter 1

The big picture: what `musrSim` is

IN THIS CHAPTER YOU WILL

- See where `musrSim` sits on top of Geant4, and why that matters.
- Understand the "configure in text, run once" philosophy that shapes everything else.
- Walk through the life of a single simulated event, from a command to a number on disk.
- Meet the muon physics that makes this toolkit special.

A muon detector is expensive, slow to build, and unforgiving of mistakes. A simulation is none of those things. Before you cut a single scintillator or book a beamline shift, you can build the whole apparatus in software, fire millions of muons at it, and read out exactly what each counter would see. That is what `musrSim` is for. This chapter is the map; the rest of the book is the territory.

1.1 Geant4, in one paragraph

Geant4 ("GEometry ANd Tracking") is a Monte-Carlo toolkit for simulating how particles move through matter. It is developed by an international collaboration — CERN, KEK, SLAC and many universities — and is the standard tool across particle, nuclear, medical and space physics. Given a geometry, a set of physics processes and some primary particles, Geant4 transports each particle step by step, deciding at every step whether it scatters, loses energy, decays or creates secondaries, and records what happens.

UNDER THE HOOD

Geant4 is a C++ *library*, not a program. Normally you write C++ classes for your detector, your primary particles and your run logic, compile them against Geant4, and run the result. `musrSim` is one such program — written once, so that you don't have to.

1.2 What `musrSim` adds

`musrSim` is a Geant4 application originally written at PSI by K. Sedlak and colleagues for muon-spin-rotation (μSR) instruments and beamlines, and since forked and extended by many groups. Its central design choice is this: *you should not have to write or compile C++ to run a new*

experiment. The detector geometry, the muon beam, the physics processes and the output are all defined in a plain-text *macro* file. One compiled executable serves every setup.

KEY POINT

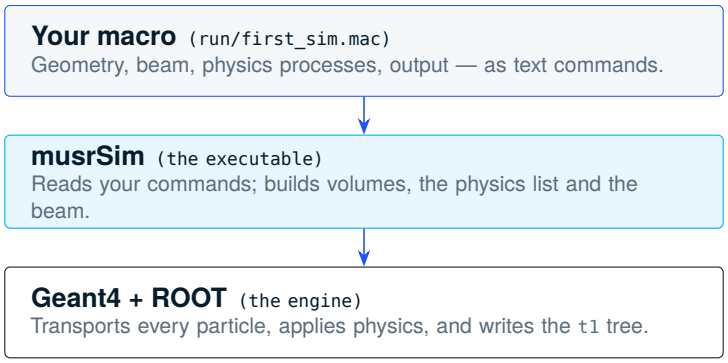
`musrSim` is not a separate simulation engine. It *is* Geant4, pre-wrapped with muon physics and a text-driven geometry builder. Everything you learn here is transferable Geant4 knowledge; `musrSim` just spares you the boilerplate.

Concretely, `musrSim` gives you:

- a library of ready-made solids you place from the macro (`box`, `tubs`, `cons`, `sphere`, and many composite shapes for real spectrometers);
- a muon-tuned physics list whose electromagnetic processes you switch on line by line, with muon decay *with spin*, muonium formation and μ^- capture available;
- a muon-beam generator (`/gun/...`) that models beam position, momentum spread, tilt and — crucially for μSR — spin polarisation, plus built-in cosmic-ray generators (EcoMug, CRY) for muography;
- a ROOT output tree with one row per event.

1.3 The three layers

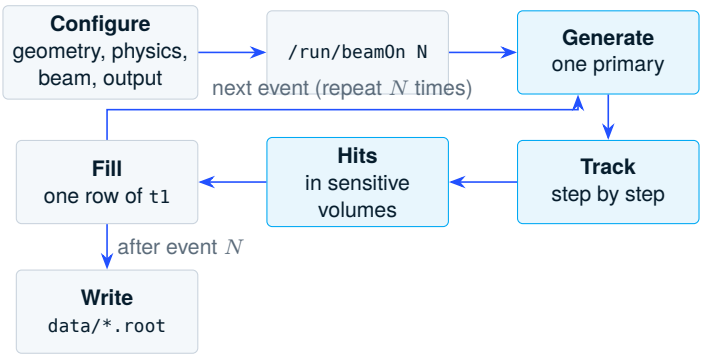
It helps to picture three layers stacked on top of each other. You write the top layer; the bottom two do the work.



When you run `../musrSim first_sim.mac`, the executable builds the geometry and physics from your macro, then executes the macro's remaining commands — the beam settings and finally `/run/beamOn`, which hands control to Geant4 to simulate the events and fill the ROOT file. You never touch the bottom two layers, but knowing they are there tells you where to look when something behaves oddly.

1.4 The life of one event

Every simulation, no matter how elaborate, follows the same loop. Fixing this picture in your head now will make Chapters 3–6 feel like variations on a theme rather than a pile of new commands.



KEY POINT

An *event* in `musrSim` is one primary particle (or, for CRY, one cosmic-ray shower) generated and tracked until it and all its secondaries leave the world or stop, with every energy deposit in a sensitive volume recorded as a *hit*. One stored event becomes one row in the output tree.

COMMON PITFALL

Unlike bare Geant4 tutorials, `musrSim` macros do *not* call `/run/initialize`. The executable initialises Geant4 itself, before your macro's commands are executed (Section 3.2 explains how). If you copied `/run/initialize` from a generic Geant4 example, remove it.

1.5 Why muons, and why this toolkit

Muons are heavy cousins of the electron ($m_\mu \approx 207\, m_e$), created abundantly in cosmic-ray showers and at accelerator beamlines. Two properties make them uniquely useful, and `musrSim` is built around both.

PHYSICS INTUITION

Muons barely notice matter. A minimum-ionising muon loses only about 2 MeV per g cm^{-2}, so it punches through metres of rock or metal in a nearly straight line. This is what makes muon radiography ("muography") possible — and what lets a few scintillator slabs act as a cosmic-ray telescope.

PHYSICS INTUITION

Muons carry a spin you can read. A polarised muon's spin precesses in a magnetic field, and its direction is imprinted on the direction of the decay positron ($\mu^+ \to e^+ \nu_e \bar{\nu}_\mu$). Measuring where the positron goes measures the field the muon sat in — the basis of μSR. This is why `musrSim` tracks muon polarisation (the `/gun/muonPolariz...` commands and the `muDecayPol...` branches) when most toolkits ignore spin entirely.

The muon's mean lifetime, $\tau_\mu = 2.197\,\mu$s, is short enough to measure on a benchtop and long enough that a stopped muon lives well past the prompt flash — a fact we exploit in Chapters 7 and 9.

1.6 What you will build in this book

By Chapter 9 you will have written, run and analysed a complete transverse-field μSR measurement: polarised muons stopping in a copper sample inside a 10 mT field, their spins precessing, and two positron counters recording an oscillating asymmetry. Fitting it recovers the field you set — in our reference run, 9.983(15) mT — end to end, from a text file to a physics result.

RECAP

- `musrSim` is a text-driven Geant4 application; you configure in a macro and never compile.
- Three layers: your macro → `musrSim` → Geant4/ROOT.
- Every event follows the same loop: generate → track → hits → one tree row.
- `musrSim` exists because muons are penetrating and spin-polarised — it tracks both.

Next: install `musrSim` and see first light →

Chapter 2

Installation and first light

IN THIS CHAPTER YOU WILL

- Install the dependencies: Geant4, ROOT, CMake and a C++ compiler.
- Get the `musrSim-upgrade` source and build the `musrSim` executable.
- Run the bundled `101.mac` and confirm you get a ROOT file.
- Look at the geometry in a 3-D viewer.

2.1 What you need

Component	Version	Role
Geant4	11.x (tested 11.4.3); 10.7 also works	Particle transport engine
ROOT	6.x (tested 6.40)	Output format and C++ analysis
CMake	$\geq$ 3.16 (CMake 4 is fine)	Build system
C++ compiler	GCC $\geq$ 9 or Clang $\geq$ 12	C++17 or newer — the build follows ROOT's standard
Python (analysis)	$\geq$ 3.9	`uproot`, `awkward`, `numpy`, `scipy`, `matplotlib`

2.2 The fastest path: one conda environment

conda-forge ships Geant4 (with its data sets), ROOT, CMake and a compiler as one consistent set. This is how the reference runs in this book were made:

Listing 2.1: Geant4 + ROOT + toolchain from conda-forge (use `mamba` or `micromamba` the same way)

```
conda create -n musrsim -c conda-forge geant4 root cmake make cxx-compiler expat
conda activate musrsim
geant4-config --version      # e.g. 11.4.3
root-config --version        # e.g. 6.40.04
```

Activating the environment also sets the Geant4 data-set variables, so there is nothing else to source. Add the Python analysis packages to the same environment (Chapter 7):

Listing 2.2: Analysis packages

```
pip install uproot awkward numpy scipy matplotlib
```

TIP

On a cluster with CVMFS you can skip installing anything: source a CERN LCG view that bundles Geant4, ROOT and a compiler, for example `source /cvmfs/sft.cern.ch/lcg/views/LCG_101/x86_64-centos7-gcc11-opt/setup.sh`. Pick a newer LCG release if your site has one.

2.3 Installing the dependencies yourself

2.3.1 Linux (Ubuntu/Debian)

Install the system libraries Geant4 needs, then build Geant4 from source. ROOT is easiest from the official binaries at `root.cern/install`.

Listing 2.3: System build dependencies

```
sudo apt-get update
sudo apt-get install -y build-essential cmake \
     libxerces-c-dev libexpat1-dev \
     libgl1-mesa-dev libglu1-mesa-dev libxmu-dev libxi-dev \
     qt6-base-dev
```

Listing 2.4: Build Geant4 11 from source

```
# after downloading and unpacking geant4-v11.x from geant4.web.cern.ch:
cd geant4-v11.4.3
mkdir build && cd build
cmake -DCMAKE_INSTALL_PREFIX=$HOME/geant4-install \
      -DGEANT4_USE_OPENGL_X11=ON -DGEANT4_USE_QT=ON \
      -DGEANT4_INSTALL_DATA=ON ..
make -j$(nproc)
make install
```

COMMON PITFALL

Note `mkdir build && cd build` — the `&&` chains the two commands. Writing "`mkdir build` and `cd build`" is not shell syntax. This is a real error we have seen copied from notes.

2.3.2 macOS

ROOT and the build tools come from Homebrew; Geant4 is not in Homebrew core, so build it from source as above — or simply use the conda environment of Section 2.2, which works the same on macOS.

Listing 2.5: macOS: tools via Homebrew, Geant4 from source

```
brew install cmake root xerces-c qt
# then build Geant4 11 from source as in the Linux section
```

2.3.3 Make the environments available

For a self-built toolchain, add these to your `~/.bashrc` or `~/.zshrc`:

Listing 2.6: Shell environment for a self-built Geant4 and a ROOT binary

```
source $HOME/geant4-install/bin/geant4.sh    # Geant4 libraries + data sets
source /path/to/root/bin/thisroot.sh         # ROOT
```

COMMON PITFALL

There is no `geant4` command and no `geant4 --version`. Geant4 exposes its settings through `geant4-config` (CMake uses it to find the libraries). If `geant4-config` is not found, the environment is not set up.

2.4 Get and build `musrSim`

Listing 2.7: Clone and build `musrSim-upgrade`

```
git clone https://github.com/kimsiang/musrSim-upgrade.git
cd musrSim-upgrade
mkdir build
cd build
cmake ../        # finds Geant4 and ROOT from your environment
make -j4         # produces the executable build/musrSim
```

`cmake` also copies every file in `run/` — the example macros, field maps and setup files — into `build/run/`, which is where you run from. If you edit a macro in `run/`, re-run `cmake ../` (or simply edit the copy in `build/run/`).

TIP

Build out of source: a separate `build/` directory keeps generated files away from the code, so `git status` stays clean and `rm -rf build` always gives you a fresh start.

2.5 First light: run the bundled example

Listing 2.8: Run your first simulation

```
cd run           # i.e. build/run
mkdir data       # output goes to data/ ; it must exist!
../musrSim 101.mac
```

You will see Geant4 print its banners, the physics processes being defined, the list of volumes with overlap checks, and finally a summary like `Number of events = 5`. The run writes `data/musrSim_jp101.root`.

KEY POINT

Output goes to `data/musrSim_jp<NNN>[_<name>].root`, where `<NNN>` is the leading integer of the macro's file name, zero-padded to three digits (`101.mac` → `jp101`; `first_sim.mac` has no number → `jp000`). The `data/` directory must already exist: without it the run stops with `!!!FATAL!!! ... Root output file ... can not be created`.

UNDER THE HOOD

The command line is `musrSim <macro> [name|idle] [seedOffset]`. Argument 1 is the macro. Argument 2, if given, becomes the `_<name>` suffix of the output file — except the special word `idle`, which instead drops you into an interactive Geant4 prompt once the macro has finished. Argument 3 is a random-seed offset for batch jobs (Section 8.4). Run without arguments, `musrSim` only prints this usage line.

`101.mac` simulates five muons and stores only events with a hit in detector 11, so its tree holds a single entry — enough to prove the chain works. Chapter 3 builds a macro of your own.

2.6 See the geometry

A simulation you cannot see is a simulation you cannot trust. The repository ships visualisation macros you call from inside a steering macro, just before `/run/beamOn`:

Listing 2.9: Write a VRML scene of the geometry and the first trajectories

```
/control/execute visVRML.mac
/run/beamOn 5
```

`visVRML.mac` opens Geant4's `VRML2FILE` driver, which writes `g4_00.wrl` (one file per run) for any VRML viewer. For a live OpenGL or Qt window instead, copy it and change its first line to `/vis/open OGL` or `/vis/open TSG_QT_GLES`; this needs a Geant4 built with those drivers. Keep `/run/beamOn` small whenever visualisation is on — every trajectory is stored. We return to visualisation in Chapter 4.

2.7 Troubleshooting the build

Symptom	Fix
`Could NOT find Geant4`	Activate the conda environment or source `geant4.sh`; or pass `-DGeant4_DIR=<prefix>/lib/cmake/Geant4`.
`root-config` not found	Source `thisroot.sh` (or activate the environment) before `cmake`; `musrSim` finds ROOT through `root-config`.
`Could NOT find EXPAT: Found unsuitable version`	CMake picked up an old system `libexpat`. Install `expat` in the conda environment and configure with `cmake -DCMAKE_PREFIX_PATH=$CONDA_PREFIX ../`.
Compile errors about missing Geant4 classes	You are on a fork older than `musrSim-upgrade`; pull the current code, which builds on Geant4 10.7–11.x.
Runs, but stops with `Root output file ... can not be created`	The `data/` directory does not exist where you launched. `mkdir data`.
Viewer window never opens	Your Geant4 was built without OpenGL/Qt; use `visVRML.mac`, or rebuild Geant4 with `-DGEANT4_USE_QT=ON`.

RECAP

- One conda-forge environment gives a matching Geant4 + ROOT + compiler; CVMFS

gives it for free on clusters.

- Build out of source: `cmake ../` then `make -j4` produces `build/musrSim` and copies `run/` to `build/run/`.
- Run `../musrSim <macro>` from `build/run`; create `data/` first; output is `data/musrSim_jp<NNN>.root`.
- `/control/execute visVRML.mac` before `/run/beamOn` gives you a 3-D picture.

Next: the macro language, and your first macro from scratch →

Chapter 3

The macro language, and your first simulation

IN THIS CHAPTER YOU WILL

- Recognise the three families of commands a `musrSim` macro is built from.
- Understand the two ways `musrSim` reads your macro — the key to every "why was my line ignored?".
- List the commands your executable understands.
- Write, run and inspect a complete simulation from a blank file.

A `musrSim` macro (also called the *steering file*) is a plain-text script. There is no compilation step: change a number, run again. Master this one file and you can build almost any instrument without touching C++.

3.1 Three families of commands

Every line in a working macro belongs to one of three families. Telling them apart is the single most useful orientation skill in this toolkit.

Family	Prefix	What it controls
Standard Geant4	`/run`, `/vis`, `/tracking`, `/control`, `/process`, `/gps`	Universal Geant4 kernel commands, identical in every Geant4 application.
`musrSim` core	`/musr/command`	Geometry, fields, physics processes, output selection. Read directly from the file by `musrSim`'s C++ (next section).
`musrSim` run control	`/musr/run`	Random seeding, progress printout.
Beam (particle gun)	`/gun`	The muon beam model: vertex, momentum, spin, decay timing, TURTLE input, and the cosmic-ray generators.

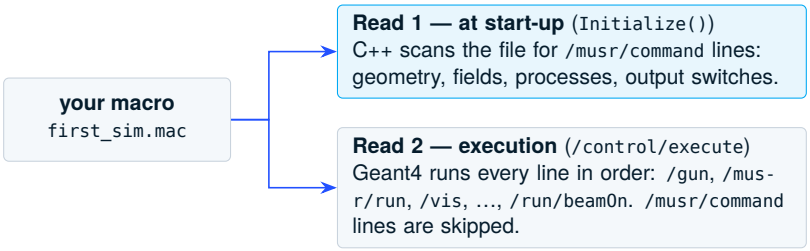
KEY POINT

The `/run`, `/vis` and `/control` commands are documented in the official Geant4 guides and never change. The `/musr/...` and `/gun/...` commands are the ones this book teaches,

because they live in your copy of the source and differ between forks. When in doubt about a `musrSim`-specific command, the source is the specification.

3.2 How `musrSim` reads a macro: twice

This is the one piece of plumbing worth understanding before you write a line. Your macro is read in two different ways.



Read 1 — start-up. `main()` first creates the detector and the physics list and calls Geant4's `Initialize()`. During that call `musrDetectorConstruction` and `musrPhysicsList` each open your macro *as a text file* and parse every line that starts with `/musr/command`, top to bottom: volumes, rotations, fields, `rootOutput` switches, `process` lines.

Read 2 — execution. Only then does `main()` run `/control/execute <yourfile>`. Now Geant4 executes the macro line by line as ordinary UI commands: `/gun/...` sets the beam, `/musr/run/...` sets seeding, `/vis/...` sets up pictures, and `/run/beamOn` starts the events. The `/musr/command` lines are passed to a messenger that deliberately ignores them — they have already done their job.

Three practical rules follow.

1. **No `/run/initialize`.** Initialisation has already happened before Read 2 begins; your macro simply ends with `/run/beamOn`.
2. **Order matters among `/musr/command` lines**: a mother volume before its daughters, a rotation matrix before the volume that uses it, a volume before the field attached to it. Where they sit relative to `/gun` lines does not matter.
3. **`/musr/command` lines must be in the macro you pass on the command line.** Read 1 does not follow `/control/execute`, so geometry or physics in an included file is silently never built. Included files are fine for `/vis` and `/gun` commands — that is exactly how `visVRML.mac` works.

UNDER THE HOOD

You can see both reads in `musrSim.cc`: `runManager->Initialize()` comes first, then `UI->ApplyCommand("/control/execute " + fileName)`. The parsing loops are `musrDetectorConstruction::Construct()` and `musrPhysicsList::ConstructEM()`; both skip lines whose first character is `#`.

3.3 List the real commands

Before trusting any command in any manual — including this one — ask your executable what it supports. Run any macro with `idle` as the second argument: once the macro has finished you get Geant4's `Idle>` prompt, where you can browse the live command tree.

Listing 3.1: Discovering `/gun` and `/musr/run` commands without reading C++

```
cd build/run
../musrSim first_sim.mac idle
# at the Idle> prompt:
#   ls /gun/                        list every beam command (and /gun/cry/, /gun/ecomug/)
#   help /gun/decaytimelimits       one command's parameters and guidance
#   ls /musr/run/                   seeding and printout commands
#   /control/manual /gun/           dump a whole subtree as text
#   exit
```

`ls` walks the command tree one level at a time; `help <command>` prints the parameters and the author's own description straight from the code; `/control/manual` dumps a subtree, which you can paste into your notes as a personal, always-correct reference.

CHECK YOUR BUILD

`/musr/command` is a *single* UI command that takes the rest of the line as a string, so `ls` cannot show its keywords (`construct`, `globalfield`, …). Those are `strcmp` branches in the C++ parsers. Appendix A lists every keyword in this build, and Section 10.2 shows how to look one up in the source.

3.4 Comments and units

Lines beginning with `#` are comments. Keep comments on their own lines: the Geant4 UI strips a trailing `# ...`, but `musrSim`'s own parsers read `/musr/command` lines with fixed field positions, and a stray word can be mistaken for a value.

Inside `/musr/command` lines the units are fixed: millimetres, degrees, tesla, nanoseconds. `/gun` commands accept an explicit unit and you should always give one: `/gun/momentum 29.8 MeV`, `/gun/vertex 0 0 -100 mm`. Being explicit in the beam block costs nothing and prevents a factor-of-ten surprise.

3.5 Your first macro, from a blank file

Here is a complete, self-contained simulation: a box of air (the mandatory `World`), one plastic-scintillator slab wired up as a detector, the electromagnetic physics, a small surface-muon beam aimed at the slab, and a line telling `musrSim` to keep only events that hit something. It ships as `run/first_sim.mac`.

Listing 3.2: `first_sim.mac` — a minimal but complete simulation

```
# ------------------------------------------------------------------
# first_sim.mac : a muon beam into a single scintillator slab
# (musrSim tutorial, Chapter 3)
```

```
# Default units: mm, ns, MeV, MeV/c, tesla, degree
# ------------------------------------------------------------

# ---- GEOMETRY ----
# construct <solid> <name> <half-x half-y half-z> <material>
#           <x y z> <motherLog> <rotation> <sensitivity> <volumeID>
/musr/command construct box World 500 500 500 G4_AIR 0 0 0 no_logical_volume norot dead -1
/musr/command construct box Slab 50 50 5 G4_PLASTIC_SC_VINYLTOLUENE 0 0 0 log_World norot musr/
  ↪ ScintSD 1

# ---- APPEARANCE (only used if you visualise) ----
/musr/command visattributes log_World invisible
/musr/command visattributes log_Slab green

# ---- PHYSICS: musrSim builds its EM physics from these lines ----
/musr/command process addDiscreteProcess gamma G4PhotoElectricEffect
/musr/command process addDiscreteProcess gamma G4ComptonScattering
/musr/command process addDiscreteProcess gamma G4GammaConversion
/musr/command process addDiscreteProcess gamma G4RayleighScattering
/musr/command process addProcess  e-  G4eMultipleScattering   -1  1  1
/musr/command process addProcess  e-  G4eIonisation           -1  2  2
/musr/command process addProcess  e-  G4eBremsstrahlung       -1  3  3
/musr/command process addProcess  e+  G4eMultipleScattering   -1  1  1
/musr/command process addProcess  e+  G4eIonisation           -1  2  2
/musr/command process addProcess  e+  G4eBremsstrahlung       -1  3  3
/musr/command process addProcess  e+  G4eplusAnnihilation      0 -1  4
/musr/command process addProcess  mu+ G4MuMultipleScattering  -1  1  1
/musr/command process addProcess  mu+ G4MuIonisation          -1  2  2
/musr/command process addProcess  mu+ G4MuBremsstrahlung      -1  3  3
/musr/command process addProcess  mu+ G4MuPairProduction      -1  4  4
/musr/command process addProcess  mu- G4MuMultipleScattering  -1  1  1
/musr/command process addProcess  mu- G4MuIonisation          -1  2  2
/musr/command process addProcess  mu- G4MuBremsstrahlung      -1  3  3
/musr/command process addProcess  mu- G4MuPairProduction      -1  4  4

# ---- OUTPUT ----
# Write data/musrSim_jp000.root ; keep only events with a detector hit
/musr/command SetOutputFileName DEFAULT
/musr/command storeOnlyEventsWithHits true
/musr/run/howOftenToPrintEvent 1000

# ---- BEAM: surface muons along +z, spin pointing along -z ----
/gun/primaryparticle mu+
/gun/vertex 0 0 -100 mm
/gun/vertexsigma 5 5 0 mm
/gun/momentum 29.8 MeV
/gun/muonPolarizVector 0 0 -1
/gun/muonPolarizFraction 1
/gun/decaytimelimits -1 -1 2197.03 ns

# ---- GO ----
/run/beamOn 20000
```

The geometry is two lines. The `World` is a 1 m cube of air (half-lengths 500 mm); the mother of the world is the literal token `no_logical_volume`. The `Slab` is a $100 \times 100 \times 10$ mm block of plastic scintillator at the origin, made sensitive by the token `musr/ScintSD` and given detector ID 1. The middle block switches on the physics, and the rest is beam and bookkeeping, which the next chapters unpack in full.

3.5.1 The physics block

COMMON PITFALL

`musrSim` builds its electromagnetic physics *from your macro*. Without the `/musr/command process` lines, photons, electrons, positrons and muons have no ionisation, scattering or annihilation at all: muons sail straight through the slab, nothing deposits energy, and the output tree is empty. We checked: `first_sim.mac` with the physics block removed stores **zero** of 20 000 events. Copy the block from `first_sim.mac` into every new macro.

Each `addProcess` line names a particle, a Geant4 process class and three ordering numbers (at-rest, along-step, post-step; `-1` means "not active"). Muon decay *with spin* is added automatically for $\mu^{\pm}$; everything else is your choice. Section 8.2 lists the processes you can add.

3.5.2 Run it

Listing 3.3: Run your first macro

```
cd build/run
mkdir -p data
../musrSim first_sim.mac      # run number 0: the file name has no leading integer
ls -l data/                   # -> musrSim_jp000.root
```

In our reference run 19 991 of the 20 000 events were stored; the handful of others left no hit and were dropped by `storeOnlyEventsWithHits`. The muons stop about 0.8 mm inside the slab's front face and decay there.

COMMON PITFALL

Three first-run failures, and their fixes. *The run stops with "can not be created"* — you forgot `mkdir -p data`. *An empty tree* — the physics block is missing, or nothing reaches your sensitive volume. *The slab is twice too big* — you typed full lengths where Geant4 wants *half*-lengths (Chapter 4 hammers this home).

PHYSICS INTUITION

Surface muons carry a momentum near 29.8 MeV/c (kinetic energy $\approx$ 4.1 MeV) and are almost 100 % spin-polarised *anti-parallel* to their momentum — the property that makes muon spin spectroscopy possible. That is why a beam travelling along $+z$ is paired with `/gun/muonPolarizVector 0 0 -1`. We return to the spin in Chapter 5.

To peek at the result before building the analysis toolchain (Chapters 6–7), open the file in ROOT:

Listing 3.4: A first look with ROOT

```
root -l data/musrSim_jp000.root
root [0] t1->Print()                // list every branch in the tree
root [1] t1->Draw("det_edep")       // energy deposited per hit (MeV)
root [2] t1->Draw("muDecayTime")    // when each muon decayed (microseconds)
root [3] .q
```

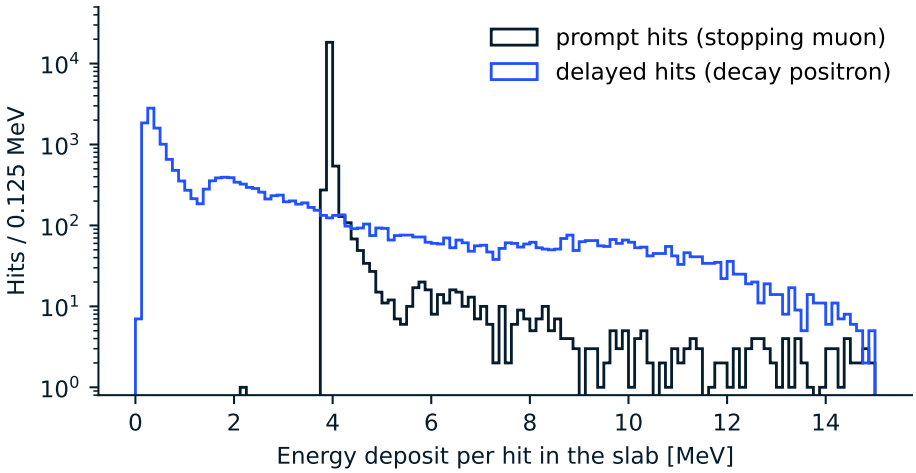


Figure 3.1: Energy per hit in the slab, from the reference run of `first_sim.mac` (log scale). Each muon produces two hits: a *prompt* one as it stops, depositing its full ≈4 MeV kinetic energy (sharp peak), and a *delayed* one, microseconds later, from its decay positron (broad spectrum). Muons that decay within the 100 ns hit-separation window merge both into one prompt hit — the prompt tail above 4 MeV.

RECAP

- Macros mix universal Geant4 commands, `musrSim`'s `/musr/command` and `/musr/run`, and the `/gun` beam.
- `/musr/command` lines are parsed from the file at start-up; everything else is executed in order afterwards. No `/run/initialize`; no `/musr/command` in included files.
- `../musrSim macro.mac idle` gives you a prompt where `ls` and `help` list the real `/gun` and `/musr/run` commands.
- A complete simulation needs a `World`, one sensitive volume, *the physics block*, a beam, and `/run/beamOn`.

Next: building real geometry with the `construct` command →

Chapter 4

Building geometry with `construct`

IN THIS CHAPTER YOU WILL

- Read and write the `/musr/command construct` line field by field.
- Never again confuse a half-length with a length.
- Nest volumes correctly using the `log_` mother convention and local coordinates.
- Rotate volumes, choose materials, import CAD, make a volume a detector, and use the special volume names.

Almost all of an instrument is geometry, and in `musrSim` almost all geometry is one command repeated: `/musr/command construct`. Each call builds one Geant4 volume — a shape, made of a material, placed inside a parent. Learn this single line thoroughly and the rest is repetition.

4.1 The construct grammar

Every `construct` call has the same field order. Here is the world volume from the bundled GPD example, `run/201.mac`, with each field numbered.

Listing 4.1: One `construct` line, dissected

```
/musr/command construct box World 2100 2100 11000 G4_AIR 0 0 0 no_logical_volume norot dead -1
#                       (1)  (2)  (3)--------------(4)    (5)-- (6)-------------- (7)-- (8)-(9)
```

(1) solid — the shape keyword: `box`, `tubs`, `cons`, `sphere`, `trd`, `para`, `torus`, `polyconeA`/`polyconeB`, `stl`, or a composite shape hard-coded in the source (Section 4.5).

(2) name — your label for this volume. Its logical volume is then referred to elsewhere as `log_<name>` (here `log_World`). Choose names carefully: some names are magic (Section 4.6).

(3) dimensions — the size parameters for that solid, in millimetres and degrees. Their number depends on the shape. For boxes and tubes these are *half*-lengths (Section 4.2).

(4) material — a Geant4/NIST material such as `G4_AIR`, `G4_Cu`, `G4_Galactic` (vacuum) or `G4_PLASTIC_SC_VINYLTOLUENE` (Section 4.5).

(5) x y z — the position of this volume's centre, in the local coordinates of its mother, in mm.

(6) mother — the logical volume that contains this one, written `log_<mother>`. The world is the sole exception: its mother is the literal token `no_logical_volume`.

(7) rotation — the name of a rotation matrix defined earlier, or `norot` (Section 4.4).

(8) sensitivity — `musr/ScintSD` to make it a detector, or `dead` for inert material (Section 4.6).

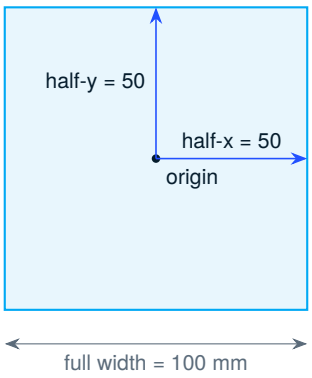
(9) volumeID — an integer that identifies this volume in the ROOT output. For detectors it becomes `det_ID`; for every volume it is the label stored in `muDecayDetID` when a muon decays there.

KEY POINT

The world volume must be named exactly `World`, must use `no_logical_volume` as its mother, and is conventionally `dead` with ID `-1`. Everything else you build lives inside it, directly or indirectly.

4.2 Half-lengths: the number-one beginner trap

For a box, the three dimensions are the *half*-widths along x, y and z — not the full edge lengths. This is inherited straight from Geant4's `G4Box`. The same convention applies to the z-extent of a `tubs` (tube) and to most other solids.



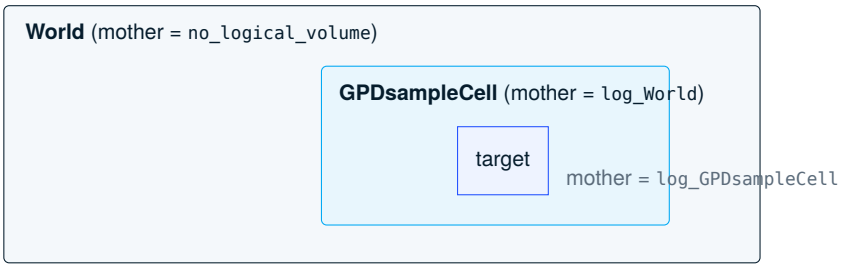
So `construct box Slab 50 50 5 ...` creates a slab that is 100 mm wide, 100 mm tall and 10 mm thick. If you mean a 100 mm cube you write `50 50 50`, not `100 100 100`.

COMMON PITFALL

Doubling errors from this one convention are the most common geometry bug in Geant4. When a detector seems twice as efficient as it should be, or a collimator hole is half the expected size, check your half-lengths first. A useful habit: write the intended full size in a comment line just above the `construct`.

4.3 Mothers, daughters, and local coordinates

Geant4 geometry is a strict hierarchy. Every volume (except the world) sits inside exactly one mother volume, and its position is given in the mother's *local* frame, with the mother's centre as origin. A daughter must fit entirely within its mother, and sibling daughters must not intersect.



To place a volume inside a mother other than the world, set field (6) to `log_<mothername>` and give its coordinates relative to that mother. In `201.mac` the copper target sits inside a sample cell, not directly in the world:

Listing 4.2: A daughter inside a non-world mother (from `201.mac`)

```
/musr/command construct tubs target 0 3.5 7 0 360 G4_Cu 0 0 0 log_GPDsampleCell norot dead 201
```

The target's centre is at $(0, 0, 0)$ in the sample cell's frame, which itself may sit deep inside the world. You cannot read a volume's global position from its `construct` line alone — follow the chain of mothers.

> **COMMON PITFALL**
>
> The mother must be constructed *before* the daughter, and referenced by its logical name `log_<mother>`, not its bare name. Writing `GPDsampleCell` instead of `log_GPDsampleCell`, or placing the daughter above the mother in the file, stops the run with a "logical volume not found" error.

4.4 Rotations

A volume is placed unrotated with the keyword `norot`. To rotate it, first define a named rotation matrix on an earlier line, then use that name in field (7). `musrSim` accepts two forms and tells them apart by the fourth number.

Axis–angle. Give a rotation axis (a_x, a_y, a_z) and a non-zero angle in degrees. From `201.mac`:

Listing 4.3: Axis–angle rotations: axis, then angle

```
# 90 deg about the x-axis; 28.81 deg about y; 45 deg about z
/musr/command rotation matrix1 1 0 0 90
/musr/command rotation matrix2 0 1 0 28.81
/musr/command rotation matrix4 0 0 1 45
```

Euler angles. Give three angles ϕ, θ, ψ in degrees and no fourth number (or a fourth number of zero):

Listing 4.4: Euler-angle rotation

```
/musr/command rotation matrix6 180 90 90
```

You then attach the matrix by name, exactly as the backward positron counters in `201.mac` do:

Listing 4.5: Using a named rotation when placing a volume

```
/musr/command construct box ScintB1 13 45 2.5 G4_PLASTIC_SC_VINYLTOLUENE 22 0 -40 log_World matrix2
  ↪ musr/ScintSD 1
```

TIP

Rotations are easy to get backwards. Define one, place one volume with it, visualise (Section 4.7), and confirm the orientation before you build ten more volumes on the same matrix.

4.5 Materials and solids

Materials are given by their Geant4/NIST names. You do not define common materials; you name them. The handful you will use constantly:

NIST name	What it is
`G4_Galactic`	Ideal vacuum (beam pipes, "save" planes).
`G4_AIR`	Air — the usual world material.
`G4_PLASTIC_SC_VINYLTOLUENE`	Plastic scintillator — muon and positron counters.
`G4_Cu`, `G4_Al`, `G4_Pb`, `G4_W`	Copper, aluminium, lead, tungsten (samples, holders, collimators).
`G4_MYLAR`, `G4_KAPTON`, `G4_POLYETHYLENE`	Windows and light materials.

About fifty extra materials are defined in `musrDetectorConstruction::DefineMaterials()` and used by their bare names: `Scintillator`, `Mylar`, `Brass`, `Steel`, `Water`, `Aerogel`, `LaBr3`, `Macor`, `MgO`, `Vacuum`, low-pressure air and helium (`Air1mbar`, `HeliumGas5mbar`, …) and — for muography — `Rock` and `Concrete`. Check that function for the exact compositions before relying on one.

Solid	Dimension fields	Notes
`box`	half-x, half-y, half-z	Rectangular block.
`tubs`	rMin, rMax, half-z, ϕ_0, $\Delta\phi$	Cylinder or tube; angles in degrees.
`cons`	rMin1, rMax1, rMin2, rMax2, half-z, ϕ_0, $\Delta\phi$	Cone / truncated cone.
`sphere`	rMin, rMax, ϕ_0, $\Delta\phi$, θ_0, $\Delta\theta$	Full or partial sphere.
`trd`	half-x1, half-x2, half-y1, half-y2, half-z	Trapezoid.
`para`	half-x, half-y, half-z, α, θ, ϕ	Parallelepiped.
`torus`	rMin, rMax, rTorus, ϕ_0, $\Delta\phi$	Torus.
`polyconeA`	ϕ_0, $\Delta\phi$, n, then three array names (z, rInner, rOuter)	Arrays defined with `arrayDef`.
`stl`	a file name instead of dimensions	CAD import (below).

Composite shapes hard-coded in the source: `GPDcollimator`, `uprofile`, `tubsbox`, `boxbox`, `cruciform`, … (Appendix C).

The beam pipe in `201.mac` shows the `tubs` field order in practice — a vacuum cylinder, radius 0 to 2005 mm, half-length 4500 mm, full azimuth:

Listing 4.6: A tube (`tubs`): rMin rMax half-z phiStart phiTotal

```
/musr/command construct tubs beampipeA 0 2005 4500 0 360 G4_Galactic 0 0 -6100 log_World norot dead
  ↪ 230
```

Polycones take their profile from named arrays, defined first with `arrayDef <name> <n> <values...>`:

Listing 4.7: A polycone from three arrays (from `201.mac`, shortened to 4 planes)

```
/musr/command arrayDef zPl  4   0.   5.   91.  105.
/musr/command arrayDef rIn  4  10.  11.   31.   31.
/musr/command arrayDef rOut 4  12.5 15.11 35.11 35.
/musr/command construct polyconeA veto 0 360 4 zPl rIn rOut G4_PLASTIC_SC_VINYLTOLUENE 0 0 24
  ↪ log_World norot musr/ScintSD 51
```

NEW IN THIS EDITION

CAD import. `construct stl <name> <file.stl> <material> <x y z> <log_mother> <rotation> <sensitivity> <volumeID>` reads a tessellated solid from an STL file through the bundled CADMesh library — handy for a real detector housing or a building in a muography study. CADMesh assumes the STL coordinates are in millimetres. Tessellated solids are slow to track through, so keep meshes modest.

UNDER THE HOOD

Composite solids such as `GPDcollimator` exist because Geant4 Boolean operations (subtracting one solid from another) cannot be expressed through the macro grammar. When you need a shape the macro cannot describe, add it to `musrDetectorConstruction.cc` and expose it as a new solid keyword — exactly how `GPDcollimator` was created — or import it with `stl`.

4.6 Making a volume a detector, and the magic names

Field (8) decides whether a volume records anything. `dead` means inert material that still scatters and stops particles but produces no hits. `musr/ScintSD` makes the volume a *sensitive detector*: energy deposits inside it become hits in the ROOT tree, tagged with the integer volumeID you gave in field (9). That integer appears in the output as `det_ID`, and it is how you will later tell one counter from another.

Listing 4.8: Five sensitive counters, distinguished only by their volumeID (from `201.mac`)

```
/musr/command construct box ScintB1 13 45 2.5 G4_PLASTIC_SC_VINYLTOLUENE  22 0 -40 log_World
  ↪ matrix2 musr/ScintSD 1
/musr/command construct box ScintB2 13 45 2.5 G4_PLASTIC_SC_VINYLTOLUENE -22 0 -40 log_World
  ↪ matrix3 musr/ScintSD 2
/musr/command construct box ScintF1 13 45 2.5 G4_PLASTIC_SC_VINYLTOLUENE -22 0  40 log_World
  ↪ matrix2 musr/ScintSD 11
/musr/command construct box ScintF2 13 45 2.5 G4_PLASTIC_SC_VINYLTOLUENE  22 0  40 log_World
  ↪ matrix3 musr/ScintSD 12
/musr/command construct box M0 5 5 1.0 G4_PLASTIC_SC_VINYLTOLUENE 0 0 -46 log_World matrix4 musr/
  ↪ ScintSD 102
```

KEY POINT

`musr/ScintSD` is the sensitive-detector type for energy deposits. You do not create ten

detector types; you create ten sensitive volumes, give each a distinct volumeID, and separate them at analysis time by selecting on `det_ID`. This is the mental model that makes the ROOT output (Chapter 6) click. (A separate optical-photon readout exists for photocathode studies; see Section 8.3.)

Some names are not just labels — `musrSim` treats them specially:

Name	Special behaviour
`World`	The mandatory outermost volume; mother is `no_logical_volume`.
`Target` or `target` (exact)	Assumed to be the sample. Muon time, polarisation and momentum on entry are stored as `muTarget...`; these branches only exist when such a volume is built.
`M0`, `M1`, `M2` (exact; or lower-case)	Typically trigger counters. Muon time and polarisation on entry stored as `muM0...`, etc.
name *containing* `save`	A tracking plane: records every particle crossing it (`save_...` branches), even in vacuum and with no energy deposit. Give it a non-zero volumeID — ID 0 is not stored.
name *starting with* `kill` or `Kill`	Any particle entering is deleted from the simulation.
name *starting with* `shield` or `Shield`	Deletes every particle *except muons* — used to speed up runs by discarding tracks that cannot reach a detector. Use with care.

COMMON PITFALL

The magic is triggered by the name. A volume you innocently call `PbSaver` or `savePlate` becomes a tracking plane ("save" matches anywhere in the name); one you call `killswitch` deletes particles. If you do not want the special behaviour, keep these words out of your names.

4.7 Seeing it, and checking for overlaps

Colour a logical volume with `visattributes` (this only affects visualisation, never physics):

Listing 4.9: Colours for visualisation

```
/musr/command visattributes log_World invisible
/musr/command visattributes log_target blue
/musr/command visattributes log_ScintB1 green
```

Available colours are `invisible`, `white`, `black`, `gray`, `red`, `darkred`, `green`, `blue`, `lightblue`, `darkblue`, `cyan`, `magenta`, `yellow`, `oxsteel`, and several house styles (`SCINT_style`, `MCP_style`, …); you may also colour by material name (`/musr/command visattributes G4_Cu red`). More can be added in `musrDetectorConstruction::SetColourOfLogicalVolume`.

Overlap checking is on in this codebase: every placement is checked as it is created, and the log shows a line `Checking overlaps for volume ... OK!` for each. An overlap corrupts particle transport without any other symptom, so never ignore a failed check.

CHECK YOUR BUILD

If a placement reports an overlap, fix the geometry before trusting a single number from the run. The usual causes are a daughter larger than its mother, a forgotten half-length, or a position that pushes a daughter through a wall. Then look at it: `/control/execute visVRML.mac` before `/run/beamOn 5` writes a 3-D scene you can rotate in any VRML viewer.

RECAP

- `construct` fields, in order: solid, name, dimensions, material, `x y z`, `log_mother`, rotation, sensitivity, volumeID.
- Box and tube dimensions are *half*-lengths; positions are in the mother's local frame.
- Rotations are axis–angle (four numbers) or Euler (three), defined before use.
- `musr/ScintSD` plus a unique volumeID makes a counter; `det_ID` separates counters later.
- The names `Target`, `M0`–`M2`, `*save*`, `kill*` and `shield*` trigger special behaviour.

Next: the muon beam, its spin, and TURTLE input →

Chapter 5

The muon beam and its spin

IN THIS CHAPTER YOU WILL

- Configure the muon beam: species, vertex, spread, momentum and direction.
- Set the muon spin — the quantity that makes muon spin spectroscopy possible.
- Control muon decay timing, and feed a realistic beam from a TURTLE file.
- Know when to reach for the General Particle Source or a cosmic-ray generator instead.

The `/gun/` commands define the primary particles: where they start, how they are spread in space and momentum, which way they point, and — uniquely for muon work — how their spins are oriented. This is the μSR beam model built into `musrSim`; it is richer than a generic particle gun because the physics of a muon beam demands it. Unlike `/musr/command` lines, `/gun` commands are ordinary Geant4 UI commands executed in order (Section 3.2), so `ls /gun/` at an `Idle>` prompt lists them all.

5.1 Species, vertex, and spread

By default the beam is positive muons. The vertex is the mean starting point; the sigmas smear it.

Listing 5.1: A spatially spread beam (from `201.mac`)

```
# default is mu+; use mu- for negative muons, or e+, e-, proton, ...
/gun/primaryparticle mu+
# mean start point, in the world frame
/gun/vertex 0 0 -1000 mm
# Gaussian spread in x and y; 0 = sharp in z
/gun/vertexsigma 25 25 0 mm
# keep vertices within radius 100 mm (and the given z range)
/gun/vertexboundary 100 -999999 999999 mm
```

A positive `vertexsigma` component draws that coordinate from a Gaussian of that standard deviation; a *negative* value $-s$ draws it uniformly in $[x_0 - s, x_0 + s]$; zero means no spread. `vertexboundary` rejects vertices outside a maximum radius and a z range, handy for forcing muons to originate inside a beam pipe.

5.2 Momentum and direction

Set the central momentum and its spread, and optionally tilt or focus the beam.

Listing 5.2: Momentum and beam optics

```
# central momentum (MeV/c) and its Gaussian sigma
/gun/momentum 100.0 MeV
/gun/momentumsmearing 3.00 MeV
# random angular divergence, and focusing proportional to the offset from the axis
/gun/tiltsigma 0.15 0.15 0 deg
/gun/pitch 0.08 deg
```

`/gun/kenergy` sets a kinetic energy instead of a momentum, and `/gun/momentumboundary` rejects momenta outside a window. `/gun/tilt` applies a fixed angle to every muon; `/gun/tiltsigma` gives a random Gaussian divergence; `/gun/pitch` applies an angle proportional to a muon's distance from the beam axis, so a positive pitch focuses the beam and a negative pitch defocuses it. To make the beam travel along a direction other than $+z$, set `/gun/direction dx dy dz` (the vector need not be normalised); the vertex spread and tilt are rotated with it.

PHYSICS INTUITION

Two momentum regimes dominate μSR. *Surface muons* at $\approx 29.8\,\mathrm{MeV/c}$ stop in the first fraction of a millimetre of a sample — ideal for thin samples. *Decay-channel muons* at higher momenta, such as the $100\,\mathrm{MeV/c}$ used in `201.mac`, penetrate deeper and suit bulky pressure cells. The momentum you choose is a physics decision about stopping depth, not a free knob.

5.3 The muon spin

This is the heart of the technique. A μSR beam is almost fully spin-polarised, and the whole measurement is watching that polarisation evolve. Set the initial spin direction as a vector; it is normalised internally.

Listing 5.3: Spin anti-parallel to a $+z$ beam (the standard μSR set-up)

```
/gun/muonPolarizVector 0 0 -1
/gun/muonPolarizFraction 1
```

`muonPolarizFraction` scales the degree of polarisation from 1 (fully aligned) through 0 (unpolarised: half the muons each way) to -1 (fully anti-aligned). Surface muons are produced with their spin pointing opposite to their momentum, which is why a $+z$ beam is paired with a $(0, 0, -1)$ spin vector.

KEY POINT

In a magnetic field the muon spin precesses, and `musrSim` transports it correctly through the field as the muon flies and after it stops. The decay positron is emitted preferentially along the spin at the moment of decay. Recording the muon polarisation at decay (`muDecayPol...`) and the decay-positron direction (`posIniMom...`) is what lets you reconstruct an asymmetry signal in Chapter 9.

5.4 Decay timing

By default Geant4 handles the muon lifetime naturally, with the PDG mean life. `/gun/decaytimelimits` lets you force decays into a chosen time window instead.

Listing 5.4: Natural decay (the window is inactive because max ≤ 0)

```
/gun/decaytimelimits -1 -1 2197.03 ns
```

The three numbers are the minimum time, maximum time and mean life. The window is used only when the maximum is positive: then every muon decays between min and max, following an exponential with the given mean life (or exactly at that time if min = max). With a non-positive maximum, as here, the line changes nothing — it is kept in the examples as a visible reminder of the default. Forced windows are useful for technical studies and misleading for physics.

5.5 A realistic beam from TURTLE

For quantitative beamline work you often want a beam that matches a real beamline, produced by the TURTLE transport code. Point `musrSim` at a TURTLE output file and it starts muons from it instead of generating them from the `/gun` distributions.

Listing 5.5: Driving the beam from a TURTLE file

```
/gun/turtlefilename my_beamline.dat
# z at which TURTLE exported the particles
/gun/turtleZ0position -1000 mm
/gun/turtleMomentumScalingFactor 1.0
```

If the run requests more muons than the file holds, the file is rewound; because the random seed keeps advancing, the reused lines yield statistically independent events. Helpers (`/gun/turtleInterpretAxes`, `/gun/turtleMomentumBite`, `/gun/turtleFirstEventNr`) adapt axis conventions, momentum spread and the starting line.

5.6 When to use the General Particle Source

For radioactive sources and other non-muon primaries, Geant4's General Particle Source (GPS) is more convenient. As soon as any line of your macro starts with `/gps/`, `musrSim` switches to the GPS generator (and adds radioactive decay for ions) and ignores the `/gun/` beam.

Listing 5.6: A radioactive ^{90}Sr source via GPS (from `102.mac`)

```
/gps/particle ion
/gps/ion 38 90 0 0
/gps/position 0 0 0
/gps/energy 0 keV
```

COMMON PITFALL

In Geant4 11, `/gps/ion` is refused unless `/gps/particle ion` comes first — older macros that omit it stop with "Illegal parameter". And GPS and `/gun/` are mutually exclusive

within a run: mixing them means your `/gun/` lines are silently ignored. For ordinary muon beams stay with `/gun/`.

5.7 Cosmic-ray muons

NEW IN THIS EDITION

For muography, shielding and detector cosmic tests, `musrSim` now includes two cosmic-ray generators that replace the beam model:

- **EcoMug** (`/gun/ecomug/...`) — samples single sea-level muons on a plane, half-sphere or cylinder, with momentum and angle cuts;
- **CRY** (`/gun/cry/...`) — generates full cosmic-ray showers (muons, neutrons, protons, gammas, electrons, pions) for a chosen date, latitude and altitude.

Appendix E shows the commands and the coordinate conventions; `run/EcoMugExample.mac` and `run/CRYExample.mac` are working starting points. In both, $+z$ is vertical *up*.

RECAP

- `/gun/vertex` and `/gun/vertexsigma` set the start point and its spread; positive sigma is Gaussian, negative is uniform.
- `/gun/momentum` with `tiltsigma`/`pitch` shapes the beam; momentum sets stopping depth.
- `/gun/muonPolarizVector` sets the spin — the observable at the centre of μSR.
- TURTLE files give realistic beams; GPS replaces `/gun/` for source decays; EcoMug and CRY give cosmic rays.

Next: sensitive detectors, hits, and the ROOT tree →

Chapter 6

Hits and the ROOT output tree

IN THIS CHAPTER YOU WILL

- Understand what a "hit" is and how many appear per event.
- Navigate the `t1` tree: event-level scalars versus per-hit arrays.
- Select a specific detector with `det_ID` — the key analysis idiom.
- Control which branches are written, and which events are kept.

When a particle deposits energy in a `musr/ScintSD` volume, `musrSim` records a *hit*. All the hits and muon information for one event become one entry in a ROOT tree named `t1`. Understanding the shape of that tree is the whole game: get it right and analysis (Chapter 7) is a few lines of Python.

6.1 What a hit is

A single event can produce several hits: more than one detector may fire, and one detector may fire more than once. Energy deposited in the same detector is summed into one hit as long as it arrives within the *signal separation time* (default 100 ns, set with `/musr/command signalSeparationTime <ns>`); a later deposit starts a new hit. That is why, in `first_sim.mac`, each stopping muon gives one prompt hit and its decay positron, microseconds later, a second one (Figure 3.1).

`musrSim` therefore stores hits as *parallel arrays*, all of the same length `det_n`, the number of hits in that event. Hit i is described by `det_ID[i]`, `det_edep[i]`, `det_time_start[i]`, and so on.



6.2 The shape of tree `t1`

The branches fall into two groups. *Event-level scalars* carry one value per event; *hit arrays* carry `det_n` values per event. The most useful of each:

Event-level scalars (one per event)	
`runID`, `eventID`	Run and event identifiers.
`weight`	Event weight (for re-weighted studies).
`muIniPosX/Y/Z`, `muIniMomX/Y/Z`	Initial muon position (mm) and momentum (MeV/c).
`muIniPolX/Y/Z`	Initial muon polarisation (spin).
`muDecayDetID`	volumeID of the volume the muon decayed in (−1 for the world).
`muDecayTime`	Decay time in µs — the muon-lifetime clock.
`muDecayPosX/Y/Z`, `muDecayPolX/Y/Z`	Position and spin at the moment of decay.
`muTargetTime`, `muTargetPolX/Y/Z`	Muon time and spin on entering a `Target` volume (only if one exists).
`muM0Time`, `muM0PolX/Y/Z`	Same, on entering an `M0` counter (also `M1`, `M2`).
`posIniMomX/Y/Z`	Initial momentum of the decay positron.
Hit arrays (length `det_n`)	
`det_n`	Number of hits in this event.
`det_ID[]`	Which detector each hit is in (your volumeID).
`det_edep[]`	Energy deposited in each hit (MeV).
`det_edep_el/pos/gam/mup/mun[]`	That energy split by e^-, e^+, γ, μ^+, μ^-.
`det_time_start[]`, `det_time_end[]`	Start and end time of each hit (µs).
`det_x/y/z[]`	Position of the first step of each hit (mm).
`det_Vrtx...[]`	Provenance of the depositing particle: vertex position, volume, process, track and particle ID.

KEY POINT

With default settings the tree has about 80 branches; naming a volume `Target`, adding `save` planes or enabling optical photons adds whole families (`muTarget...`, `save_...`, `odet_...`), up to a few hundred. You do not need most of them. For a typical positron-counter analysis four branches carry the physics: `det_n`, `det_ID`, `det_edep` and `det_time_start`, plus `muDecayTime` for lifetimes. Appendix B lists them all.

COMMON PITFALL

A value of −1000 (sometimes −999) is `musrSim`'s "not defined for this event" marker, not a measurement: `muTargetTime` is −1000 when the muon never reached the sample, and the muon branches stay −1000 when the primary is not a muon. Always cut these sentinel values before histogramming or fitting.

6.3 Selecting a detector with `det_ID`

Because every counter writes into the same `det_edep` array, you isolate one counter by selecting on its `det_ID`. This single idiom underlies nearly all `musrSim` analysis. In a ROOT session on the output of `201.mac`:

Listing 6.1: Per-detector spectra in ROOT, selected by `det_ID`

```
root -l data/musrSim_jp201.root
root [0] t1->Draw("det_edep", "det_ID==1")                // backward counter B1
root [1] t1->Draw("det_edep", "det_ID==11")               // forward counter F1
root [2] t1->Draw("muDecayTime", "muDecayDetID==201")     // muons that died in the target
root [3] t1->Scan("det_ID:det_edep:det_time_start", "", "colsize=12")
```

The second argument to `Draw` is a selection evaluated *per hit*, so `"det_ID==11"` keeps only hits in the counter you labelled 11.

6.4 Controlling the output

Two kinds of control matter: which events are saved, and which branches are written.

Which events. By default only events with at least one detector hit are kept, since empty events cannot contribute to a real measurement. You can change this:

Listing 6.2: Event-selection switches (pick what you need)

```
# default true: keep only events with a hit in any detector
/musr/command storeOnlyEventsWithHits true
# keep only events with a hit in detector 11
/musr/command storeOnlyEventsWithHitInDetID 11
# keep only events whose muon decayed in volume 201
/musr/command storeOnlyEventsWithMuonsDecayedInDetID 201
```

(`storeOnlyTheFirstTimeHit true` also exists; it stops each track at its first hit and is meant for special studies only.)

Which branches. Trim the tree by switching branches off; `201.mac` does this for the optical and vertex families it does not need. A few branches are off by default and must be switched *on*:

Listing 6.3: Pruning unused branches keeps files small and fast

```
/musr/command rootOutput odet_ID off
/musr/command rootOutput det_VvvKine off
# off by default -- switch on if needed (fills BxIntegral)
/musr/command rootOutput fieldIntegralBx on
```

CHECK YOUR BUILD

The switch name is usually the branch name, but not always: `det_start`/`det_end` control `det_time_start`/`det_time_end`, and `fieldIntegralBx` fills `BxIntegral`. Only the field integrals, `phot_time`, `odet_timeC1` and `odet_timeCFDarray` accept `on`. When unsure, search `musrDetectorConstruction.cc` for `"rootOutput"`.

Naming the file. The output path is `<dir>/musrSim_jp<NNN>[_<name>].root`. `<NNN>` is the leading integer of the macro's file name, zero-padded to three digits; `<name>` comes from the second command-line argument or, if none is given, from `SetOutputFileName`; `<dir>` is `data` unless you change it.

Listing 6.4: Naming the ROOT file from inside the macro

```
# -> data/musrSim_jp<NNN>.root
/musr/command SetOutputFileName DEFAULT
# or give a suffix: -> data/musrSim_jp<NNN>_myRun.root
```

```
# /musr/command SetOutputFileName myRun
# write somewhere other than data/ (the directory must exist)
# /musr/command rootOutputDirectoryName /scratch/me/sims
```

TIP

Prune first, simulate second. Deciding which branches you need before a million-event run can shrink the output several-fold and speed up both writing and later reading. You can always switch a branch back on and re-run a small job to check it.

RECAP

- One stored event → one entry in tree `t1`; hits are parallel arrays of length `det_n`, split in time by `signalSeparationTime`.
- Event-level scalars (`muDecayTime`, `muIniPol...`) versus per-hit arrays (`det_edep[]`).
- Isolate a counter by selecting on `det_ID`; cut the −1000 sentinels.
- `storeOnly...` chooses which events are kept; `rootOutput <branch> off` prunes the tree.

Next: analysing the tree in Python →

Chapter 7

Analysing the tree in Python

IN THIS CHAPTER YOU WILL

- Open a `musrSim` ROOT file in Python with `uproot` — no ROOT installation required.
- Handle the per-hit jagged arrays correctly with `awkward`.
- Plot an energy spectrum and fit the muon lifetime.
- Package the boilerplate into a small, reusable loader.

You can analyse `musrSim` output entirely in Python. The `uproot` library reads ROOT files without a ROOT installation, and `awkward` gives you fast, NumPy-like operations on the variable-length hit arrays. This chapter builds a complete analysis of the `first_sim.mac` output in small pieces. The finished scripts live in `doc/scripts/`.

7.1 Opening the file

Install the toolkit once (Section 2.2), then open the tree by name.

Listing 7.1: Reading tree `t1` into arrays

```
import uproot
import awkward as ak
import numpy as np

t = uproot.open("data/musrSim_jp000.root")["t1"]    # the tree is always called t1
print(t.num_entries, "events,", len(t.keys()), "branches")

# Per-hit branches come back as jagged awkward arrays,
# event-level branches as flat arrays.
data = t.arrays(["det_n", "det_ID", "det_edep", "det_time_start",
                 "muDecayTime", "muDecayDetID"], library="ak")
```

For the reference run this prints `19991 events, 80 branches`.

KEY POINT

`det_edep` is *jagged*: event 0 might have two hits and event 1 one. You cannot treat it as a flat column until you either flatten it or select one hit per event. Reaching for `np.array(det_edep)` on a jagged branch is the first mistake most people make — use `awkward` instead.

7.2 Energy spectra: flattening and masking

To histogram the energy of every hit, flatten the jagged array. To histogram one counter, mask on `det_ID` *before* flattening, so the mask and the data stay aligned.

Listing 7.2: All-hit spectrum, and a single-counter spectrum

```
import matplotlib.pyplot as plt

all_edep  = ak.flatten(data.det_edep)              # 1-D array of all hits
mask      = data.det_ID == 1                        # jagged boolean, same shape as det_edep
edep_slab = ak.flatten(data.det_edep[mask])         # mask, THEN flatten

fig, ax = plt.subplots(figsize=(6, 4))
ax.hist(np.asarray(all_edep),  bins=100, range=(0, 15), histtype="step", label="all hits")
ax.hist(np.asarray(edep_slab), bins=100, range=(0, 15), histtype="step", label="det_ID == 1")
ax.set_xlabel("Energy deposit [MeV]"); ax.set_ylabel("Hits / bin"); ax.legend()
fig.savefig("edep_spectrum.png", dpi=150)
```

COMMON PITFALL

Order matters: `ak.flatten(det_edep)[mask]` is wrong, because once `det_edep` is flattened the per-event mask no longer lines up with it. Always apply the jagged mask first (`det_edep[mask]`) and flatten the result.

To separate the prompt muon hits from the delayed positron hits of Figure 3.1, mask on time in exactly the same way: `data.det_edep[data.det_time_start < 0.01]` keeps hits that started within 10 ns.

7.3 The muon lifetime

`muDecayTime` is an event-level scalar (in µs), so no flattening is needed — but you must cut the sentinel for muons without a recorded decay, and you will usually select muons that stopped in a particular volume.

Listing 7.3: Histogram and fit the muon decay time

```
from scipy.optimize import curve_fit

t_us = np.asarray(data.muDecayTime)
good = np.asarray((data.muDecayDetID == 1) & (data.muDecayTime > 0))   # stopped in the slab
t_us = t_us[good]

counts, edges = np.histogram(t_us, bins=100, range=(0, 15))
centres = 0.5 * (edges[:-1] + edges[1:])

def decay(t, N0, tau, bkg):
    return N0 * np.exp(-t / tau) + bkg

popt, pcov = curve_fit(decay, centres, counts, p0=[counts.max(), 2.2, 1.0],
                       sigma=np.sqrt(counts + 1), absolute_sigma=True)
tau, dtau = popt[1], np.sqrt(pcov[1, 1])
print(f"tau = {tau:.4f} +/- {dtau:.4f} us   (PDG: 2.1970 us)")
```

On the 19 990 muons of the reference run this gives $\tau = 2.214(20)\,\mu s$, within one standard deviation of the PDG value (Figure 7.1).

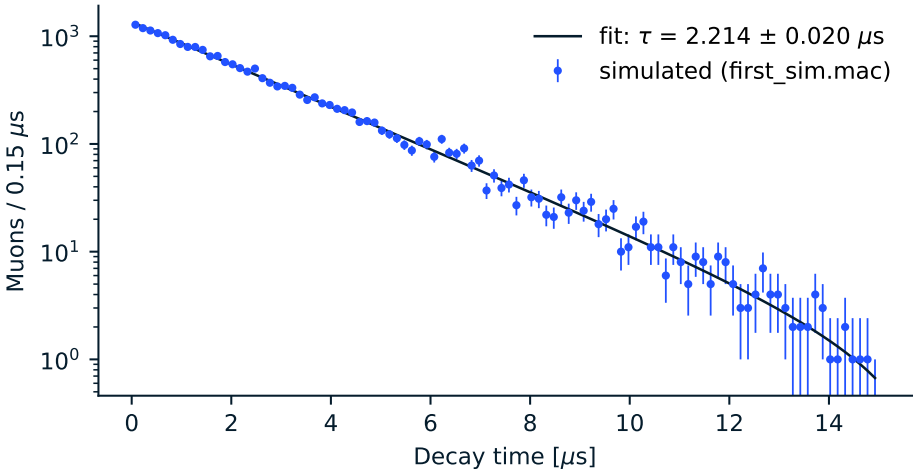


Figure 7.1: Decay-time distribution of the muons stopped in the slab in `first_sim.mac`, with the exponential-plus-constant fit.

> **PHYSICS INTUITION**
>
> A pure positive-muon sample recovers the free mean life, $\tau_{\mu^+} \approx 2.197\,\mu s$. Fit a negative-muon run (`/gun/primaryparticle mu-`, with `G4MuonMinusCapture` in the physics block) and you will measure something shorter: a μ^- stopped in matter can be captured by a nucleus, so its disappearance rate is the sum of decay and capture, and the shift depends strongly on Z of the stopping material. Seeing that shift emerge from your own simulation is a satisfying check that the physics list is doing its job.

The complete version of this analysis, which also produces both plots, is `doc/scripts/first_analysis.py`; run it from `build/run` as `python ../../doc/scripts/first_analysis.py`.

7.4 A reusable loader

Wrap the boilerplate once so every later analysis starts from a clean function.

Listing 7.4: `doc/scripts/load.py` — a small, reusable entry point

```python
"""load.py -- a small, reusable entry point for musrSim output (tutorial, Chapter 7)."""
import uproot, awkward as ak, numpy as np

DEFAULT_BRANCHES = [
    "det_n", "det_ID", "det_edep", "det_time_start",
    "muDecayTime", "muDecayDetID",
    "muDecayPolX", "muDecayPolY", "muDecayPolZ",
    "posIniMomX", "posIniMomY", "posIniMomZ",
]


def load(path, tree="t1", branches=DEFAULT_BRANCHES):
    """Open a musrSim file and return an awkward record array."""
```

```
    return uproot.open(path)[tree].arrays(branches, library="ak")


def counter(data, det_id, field="det_edep"):
    """Flatten one per-hit field for a single detector ID."""
    return ak.flatten(data[field][data.det_ID == det_id])


if __name__ == "__main__":
    import sys
    d = load(sys.argv[1])
    print(d.type, "\n", len(d), "events")
```

TIP

For a whole run directory, `uproot.concatenate("data/musrSim_jp201_job*.root:t1", branches, library="ak")` reads and stitches many files at once — the natural next step when you split a big job across a batch farm (Section 8.4).

RECAP

- `uproot` opens `t1` with no ROOT install; per-hit branches are jagged `awkward` arrays.
- Mask on `det_ID` (or time) before flattening; flatten only when you want a 1-D histogram.
- `muDecayTime` fits to $\approx 2.197\,\mu$s for μ^+; cut the sentinels first.
- A tiny `load()` / `counter()` pair removes the boilerplate from every future script.

Next: fields, the physics list, and running at scale →

Chapter 8

Fields, physics, and running at scale

IN THIS CHAPTER YOU WILL

- Add magnetic and electric fields — uniform, mapped from a file, and quadrupole.
- Understand exactly which physics your run contains, and how to change it.
- Enable optical-photon tracking, and know what it costs.
- Split a large simulation across many jobs with statistically independent random numbers.

This chapter collects the capabilities you reach for once the basics work: fields (essential for real μSR), the physics list, optical photons, and the mechanics of large production runs.

8.1 Magnetic and electric fields

Fields are added with the `globalfield` command family and attached to a logical volume. Fields from several elements may overlap; `musrSim` sums them.

Uniform field. Give the half-dimensions of the box the field fills, the keyword `uniform`, the box centre, the host logical volume, then *six* numbers: $B_x\, B_y\, B_z$ in tesla and $E_x\, E_y\, E_z$ in kV/mm.

Listing 8.1: A uniform 10 mT field along x, over a 800 mm cube at the origin

```
# globalfield <name> <halfX halfY halfZ> uniform <x y z> <logVol> <Bx By Bz> <Ex Ey Ez>
/musr/command globalfield TF 400 400 400 uniform 0 0 0 log_World 0.01 0 0 0 0 0
```

COMMON PITFALL

Always give all six field components, even when $E = 0$. The parser reads exactly six numbers; leaving the electric part off leaves those values undefined.

Field map. A mapped field reads a table from a file and scales it, as the GPD solenoid does in `201.mac`. The first three numbers are the map's centre; then `fromfile`, the table format, the file, the host volume and the field value in tesla:

Listing 8.2: A solenoid field from a 2-D field map (from `201.mac`)

```
# globalfield <name> <x y z> fromfile <format> <tablefile> <logVol> <value[T]> [<final> <nSteps>]
/musr/command globalfield centralSolenoidField 0. 0. 0. fromfile 2D GPD_coil_primitive.table
  ↪ log_target 0.03
```

The format names the table layout: `2D` or `3D` (OPERA-style tables), `2DB/2DE` and `3DB/3DE` (magnetic/electric maps in the PSI format), plus a few specialised variants listed in `musrTabulatedElementField.cc`. Supplying the two optional trailing numbers — a final value and a number of steps — ramps the field linearly with event number, a compact way to simulate a field scan in a single run.

Quadrupole. `globalfield <name> <x y z> quadrupole <halfLength> <radius> <fringeFactor> <logVol> <gradient[T/m]>` builds a magnetic quadrupole with fringe fields.

Check what you built, and tune the integration accuracy if needed:

Listing 8.3: Verifying the field and tuning the stepping accuracy

```
/musr/command globalfield printFieldValueAtPoint 0 0 0
/musr/command globalfield printparameters
/musr/command globalfield setparameter SetLargestAcceptableStep 200
/musr/command globalfield setparameter SetMinimumEpsilonStep 5e-5
/musr/command globalfield setparameter SetMaximumEpsilonStep 0.001
/musr/command globalfield setparameter SetDeltaOneStep 0.1
```

`printFieldValueAtPoint` prints, just before the first event, a line such as
`Field at (0.00, 0.00, 0.00) mm is:  B = (0.01, 0, 0) T,  E = (0, 0, 0) kV/mm.`

CHECK YOUR BUILD

Always print the field at a few points after defining a mapped field. A map with the wrong axis convention, units or centre offset is the most common silent error in field work, and the printout catches it in seconds. Note also that Geant4 11 refuses a maximum epsilon step above 0.01; `musrSim` now lowers larger values to 0.01 and tells you so.

KEY POINT

In a field, `musrSim` transports the muon spin along with its trajectory — and keeps precessing it after the muon stops. The recorded `muDecayPol...` reflects all the precession the muon underwent, so a transverse field produces the oscillating asymmetry you will build in Chapter 9.

8.2 The physics list, exactly

`musrSim` always uses its own physics list, `musrPhysicsList`. Most of it is assembled from your macro (Section 3.5.1); a few pieces are fixed in C++:

Always present	From your macro (`/musr/command process ...`)
Transportation; muon decay *with spin* (`G4DecayWithSpin`) for $\mu^{\pm}$; multiple scattering and ionisation for other charged hadrons and ions; muonium (`Mu`) scattering and decay; radioactive decay for ions when GPS is used.	**addProcess** (particle, process, three ordering numbers): `G4eMultipleScattering`, `G4eIonisation`, `G4eBremsstrahlung`, `G4eplusAnnihilation`, `G4MuMultipleScattering`, `G4MuIonisation`, `G4MuBremsstrahlung`, `G4MuPairProduction`, `musrMuFormation`, `musrMuEnergyLossLandau`, `G4MuonMinusCapture`, `G4Scintillation`, `G4Cerenkov`, `G4StepLimiter`, `G4UserSpecialCuts`. **addDiscreteProcess**: `G4PhotoElectricEffect`, `G4ComptonScattering`, `G4GammaConversion`, `G4GammaConversionToMuons`, `G4RayleighScattering`, `G4CoulombScattering`, `G4OpAbsorption`, `G4OpRayleigh`, `G4OpBoundaryProcess`, `G4OpWLS`.

Production cuts default to 0.1 mm for photons and electrons and can be changed with `/musr/command process cutForGamma <mm>` and `cutForElectron <mm>`.

Two consequences follow. First, *there is no hadronic physics* for nucleons and pions beyond ionisation and scattering, and no photo- or electro-nuclear reactions: `musrSim` is an electromagnetic-plus-muon toolkit. If neutron or pion interactions matter to you, that is a C++ change. Second, every example macro's `process` block is meaningful: it is your physics.

> **COMMON PITFALL**
>
> An unknown process name stops the run with `not implemented in musrPhysicsList.cc`. Older macros sometimes name processes that were never wired in; add the one line to `musrPhysicsList::ConstructEM()` — this edition added `G4GammaConversionToMuons` exactly that way — or remove the line. (`/musr/command G4EmExtraPhysics` is still accepted for compatibility, but does nothing.)

For μ^+ stopping in matter, muonium formation can be modelled with `musrMuFormation`; for μ^-, add `G4MuonMinusCapture` as an at-rest process:

Listing 8.4: Negative-muon capture at rest

```
/musr/command process addProcess mu- G4MuonMinusCapture 0 -1 -1
```

8.3 Optical photons

For studies of light yield and photodetector timing you can track individual scintillation and Cherenkov photons. It is off by default because it is expensive. Turning it on takes four ingredients, all in the macro; `run/110.mac` is a complete working example.

1. `/musr/command G4OpticalPhotons true`.
2. Optical properties for each material photons travel in — a named `materialPropertiesTable`, attached with `setMaterialPropertiesTable`: refractive index, absorption length, emission spectrum, yield, time constants.
3. The optical processes: `G4Scintillation` (and/or `G4Cerenkov`) for the charged particles, and `G4OpAbsorption`, `G4OpRayleigh`, `G4OpBoundaryProcess` for `opticalphoton`.
4. Optical surfaces (`opticalSurface`) with an `EFFICIENCY` where photons are detected, e.g. a

scintillator–APD boundary.

Listing 8.5: Optical properties of a plastic scintillator (shortened from `110.mac`; photon energies in MeV)

```
/musr/command G4OpticalPhotons true
/musr/command materialPropertiesTable optScintMPT RINDEX 2  2.695e-6 3.64618e-6  1.58 1.58
/musr/command materialPropertiesTable optScintMPT SCINTILLATIONYIELD 0 9167.
/musr/command materialPropertiesTable optScintMPT FASTTIMECONSTANT   0 1.32
/musr/command materialPropertiesTable optScintMPT YIELDRATIO         0 1.0
/musr/command setMaterialPropertiesTable optScintMPT G4_PLASTIC_SC_VINYLTOLUENE
/musr/command process addProcess e- G4Scintillation 4 -1 4
/musr/command process addDiscreteProcess opticalphoton G4OpAbsorption
/musr/command process addDiscreteProcess opticalphoton G4OpBoundaryProcess
```

When active, the `odet_...` branches record photon counts and arrival times per optical detector, and the `OPSA` command family simulates the photodetector signal.

CHECK YOUR BUILD

Geant4 11 renamed the two-component scintillation properties (`FASTCOMPONENT` → `SCINTILLATIONCOMPONENT1`, `FASTTIMECONSTANT` → `SCINTILLATIONTIMECONSTANT1`, `YIELDRATIO` → `SCINTILLATIONYIELD1/2`, and the `SLOW...` equivalents). `musrSim` translates the old names automatically, so both spellings work in your macro.

COMMON PITFALL

Optical photons can multiply the number of tracked particles per event by thousands, slowing a run by one to two orders of magnitude. Turn them on only for the detector study that needs them, keep the event count modest, and leave them off for geometry, beam and asymmetry work.

8.4 Running at scale

Physics runs need millions of events, which means many jobs. Each job must use *different random numbers* and write a *differently named* file. The second is easy — the second command-line argument sets the file suffix. The first needs care.

KEY POINT

How the random engine is seeded is set by `/musr/run/randomOption`:

- `0` (default) — no seeding: every run of the same macro produces the *same* events;
- `1` — seed from the clock *plus* 100 × the seed offset (third command-line argument or `SetRndSeedOffset`);
- `2` — re-seed each event with its event number: perfectly reproducible, event by event, and therefore *identical* across jobs;
- `5` — a fixed seed from `/musr/command SetRndSeed <n>` (fully reproducible run).

For independent parallel jobs use **option 1** with a different offset per job.

We tested this directly: two 300-event jobs of `first_sim.mac` with seed offsets 1 and 2 produced

identical muon decay times under options 0 and 2, and different ones under option 1.

Listing 8.6: Settings for production jobs

```
/musr/run/randomOption 1
/musr/run/howOftenToPrintEvent 10000
# wall-clock guard in seconds: stop the run cleanly before the batch limit
/musr/command maximumRunTimeAllowed 82800
```

Listing 8.7: A simple job split, then merge with ROOT's `hadd`

```
for i in $(seq 1 50); do
  ../musrSim 201.mac job${i} ${i} > log_job${i}.txt 2>&1 &
done
wait
# outputs: data/musrSim_jp201_job1.root ... data/musrSim_jp201_job50.root
hadd data/musrSim_jp201_merged.root data/musrSim_jp201_job*.root
```

On an HTCondor farm the same idea becomes a submit file with `Queue 50`, passing `$(Process)` as both the name suffix and the seed offset.

COMMON PITFALL

Option 1 seeds from the clock, so a job is not reproducible after the fact unless you note its seeds: they are printed at the start of the log (`seed1: ...; seed2: ...`). Keep the logs. If you need bit-exact reproducibility of a single job, use option 5 with an explicit `SetRndSeed` per job instead.

TIP

Two run-time safety valves. `maximumRunTimeAllowed` defaults to 85 000 s (about 23.6 hours): a longer job stops itself there, writing what it has — raise it for multi-day runs. And to stop a running job gracefully, create a file named after its macro with `.stop` instead of `.mac` (`touch 201.stop`): the run ends after the current event and the ROOT file is written normally.

TIP

Merge with `hadd` when you want a single ROOT file; for Python analysis you can skip the merge: `uproot.concatenate("data/musrSim_jp201_job*.root:t1", ...)` reads the whole set directly.

RECAP

- `globalfield ... uniform` needs six field numbers; `... fromfile <format>` reads a map; always `printFieldValueAtPoint`.
- `musrPhysicsList` is the only physics list; your `process` lines are your EM physics; there is no hadronic physics.
- `G4OpticalPhotons true` plus material tables, processes and surfaces enables photon tracking — at a large speed cost.

- Parallel jobs need `randomOption 1` and a different seed offset each; merge with `hadd` or read all files in `uproot`.

Next: a complete μSR asymmetry measurement →

Chapter 9

Capstone: a transverse-field asymmetry measurement

IN THIS CHAPTER YOU WILL

- Assemble geometry, field, physics, beam and output into one simulation.
- Reproduce muon-spin precession in a transverse magnetic field.
- Build the asymmetry signal and fit a precession frequency.
- Recover the field you set from the frequency you measured.

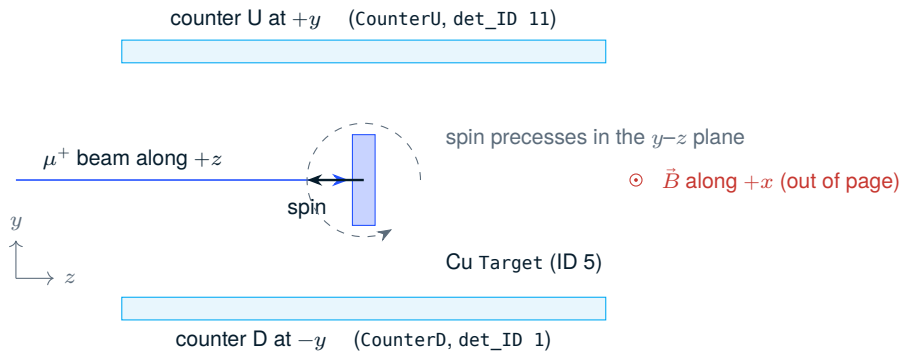
Everything so far has been groundwork. Here we run a complete, if idealised, transverse-field muon spin rotation experiment: muons stop in a sample sitting in a magnetic field perpendicular to their spin, their spins precess, and the decay positrons are counted in two opposing detectors. The normalised difference of those counts oscillates at the Larmor frequency — and from it we recover the field.

9.1 The physics in one paragraph

A muon spin in a magnetic field B precesses at the Larmor frequency $f = (\gamma_\mu/2\pi)\,B$, with $\gamma_\mu/2\pi \approx 135.54\,\mathrm{MHz/T}$. The decay positron is emitted preferentially along the muon spin at the instant of decay. If the spin starts along the beam axis (z) and the field points along x, the spin rotates in the y–z plane, so its y-component swings as $\sin(2\pi f t)$. Two counters facing each other along $\pm y$ therefore see their positron rates rise and fall in antiphase. The asymmetry

$$A(t) = \frac{N_\mathrm{D}(t) - N_\mathrm{U}(t)}{N_\mathrm{D}(t) + N_\mathrm{U}(t)}$$

cancels the exponential decay common to both counters and leaves a clean oscillation.



9.2 The macro

The geometry is a copper sample named `Target` and two scintillator plates, plus a uniform transverse field over the whole world and the standard physics block. The sample's name switches on the `muTarget...` branches, and the run keeps only muons that decayed inside it. The macro ships as `run/tf_musr.mac`:

Listing 9.1: `tf_musr.mac` — a complete transverse-field experiment

```
# ----------------------------------------------------------------
# tf_musr.mac : transverse-field muSR asymmetry
# (musrSim tutorial, Chapter 9)
# Units: mm, ns, MeV, MeV/c, tesla, degree
# ----------------------------------------------------------------

# ---- GEOMETRY ----
/musr/command construct box World 400 400 400 G4_AIR 0 0 0 no_logical_volume norot dead -1
# sample the muons stop in; the name "Target" fills the muTarget* branches
/musr/command construct box Target 15 15 3 G4_Cu 0 0 0 log_World norot dead 5
# two positron counters facing each other along +y and -y
/musr/command construct box CounterU 40 3 40 G4_PLASTIC_SC_VINYLTOLUENE 0 45 0 log_World norot musr
  ↪ /ScintSD 11
/musr/command construct box CounterD 40 3 40 G4_PLASTIC_SC_VINYLTOLUENE 0 -45 0 log_World norot
  ↪ musr/ScintSD 1

/musr/command visattributes log_World invisible
/musr/command visattributes log_Target blue
/musr/command visattributes log_CounterU green
/musr/command visattributes log_CounterD green

# ---- TRANSVERSE FIELD: 10 mT along x, over the whole world ----
# globalfield <name> <halfX halfY halfZ> uniform <x y z> <logVol> <Bx By Bz> <Ex Ey Ez>
/musr/command globalfield TF 400 400 400 uniform 0 0 0 log_World 0.01 0 0 0 0 0
/musr/command globalfield printFieldValueAtPoint 0 0 0

# ---- PHYSICS: musrSim builds its EM physics from these lines ----
/musr/command process addDiscreteProcess gamma G4PhotoElectricEffect
/musr/command process addDiscreteProcess gamma G4ComptonScattering
/musr/command process addDiscreteProcess gamma G4GammaConversion
/musr/command process addDiscreteProcess gamma G4RayleighScattering
/musr/command process addProcess  e-  G4eMultipleScattering   -1  1  1
/musr/command process addProcess  e-  G4eIonisation           -1  2  2
/musr/command process addProcess  e-  G4eBremsstrahlung       -1  3  3
/musr/command process addProcess  e+  G4eMultipleScattering   -1  1  1
```

```
/musr/command process addProcess  e+  G4eIonisation           -1  2  2
/musr/command process addProcess  e+  G4eBremsstrahlung       -1  3  3
/musr/command process addProcess  e+  G4eplusAnnihilation      0 -1  4
/musr/command process addProcess  mu+ G4MuMultipleScattering  -1  1  1
/musr/command process addProcess  mu+ G4MuIonisation          -1  2  2
/musr/command process addProcess  mu+ G4MuBremsstrahlung      -1  3  3
/musr/command process addProcess  mu+ G4MuPairProduction      -1  4  4
/musr/command process addProcess  mu- G4MuMultipleScattering  -1  1  1
/musr/command process addProcess  mu- G4MuIonisation          -1  2  2
/musr/command process addProcess  mu- G4MuBremsstrahlung      -1  3  3
/musr/command process addProcess  mu- G4MuPairProduction      -1  4  4

# ---- OUTPUT ----
/musr/command SetOutputFileName tf
/musr/command storeOnlyEventsWithMuonsDecayedInDetID 5
/musr/run/howOftenToPrintEvent 50000

# ---- BEAM: surface muons along +z, spin anti-parallel (-z) ----
/gun/primaryparticle mu+
/gun/vertex 0 0 -80 mm
/gun/vertexsigma 3 3 0 mm
/gun/momentum 29.8 MeV
/gun/muonPolarizVector 0 0 -1
/gun/muonPolarizFraction 1
/gun/decaytimelimits -1 -1 2197.03 ns

# ---- GO ----
/run/beamOn 200000
```

Listing 9.2: Run it (about 1.5 minutes on one core)

```
cd build/run
mkdir -p data
../musrSim tf_musr.mac        # -> data/musrSim_jp000_tf.root (suffix from SetOutputFileName)
```

The log confirms the field just before the first event: `Field at (0.00, 0.00, 0.00) mm is: B = (0.01, 0, 0) T`.

PHYSICS INTUITION

Surface muons at 29.8 MeV/c stop within a fraction of a millimetre, so nearly all of them come to rest inside the 6 mm-thick copper sample and decay there. The line `storeOnlyEventsWithMuonsDecayedInDetID 5` discards the few that stop elsewhere, so the analysis sees a clean sample-stopped population regardless of the exact stopping profile. Copper is also a good choice physically: muons in a non-magnetic metal do not form muonium and keep their full polarisation.

9.3 The analysis

Load the tree, keep sample-stopped muons, and split the decay times by which counter caught the positron. Events in which *both* counters fired are ambiguous and dropped. The exponential lifetime divides out of the asymmetry, leaving the precession.

Listing 9.3: `doc/scripts/asymmetry.py` — from ROOT file to Larmor frequency

```python
"""asymmetry.py -- from a musrSim ROOT file to a Larmor frequency (tutorial, Chapter 9).

Usage:  python asymmetry.py [data/musrSim_jp000_tf.root]
"""
import sys
import uproot, awkward as ak, numpy as np
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
from scipy.optimize import curve_fit

path = sys.argv[1] if len(sys.argv) > 1 else "data/musrSim_jp000_tf.root"
d = uproot.open(path)["t1"].arrays(
    ["det_ID", "muDecayTime", "muDecayDetID"], library="ak")

# keep muons that stopped and decayed in the sample (volumeID 5); cut sentinels
sel = (d.muDecayDetID == 5) & (d.muDecayTime > 0)
d = d[sel]
t = np.asarray(d.muDecayTime)                       # microseconds

# per event: did the up (+y) or down (-y) counter register a hit?
hitU = np.asarray(ak.any(d.det_ID == 11, axis=1))
hitD = np.asarray(ak.any(d.det_ID == 1, axis=1))
both = hitU & hitD                                  # ambiguous: drop
hitU, hitD = hitU & ~both, hitD & ~both

bins = np.linspace(0, 10, 201)
NU, _ = np.histogram(t[hitU], bins=bins)
ND, _ = np.histogram(t[hitD], bins=bins)
centre = 0.5 * (bins[:-1] + bins[1:])

tot = NU + ND
ok = tot > 20                                       # ignore starved bins at late time
A = (ND[ok] - NU[ok]) / tot[ok]                     # the lifetime cancels in the ratio
Aerr = np.sqrt((1 - A**2) / tot[ok])
tg = centre[ok]

def osc(t, A0, f, phi, lam):                        # damped precession
    return A0 * np.exp(-lam * t) * np.sin(2 * np.pi * f * t + phi)

popt, pcov = curve_fit(osc, tg, A, p0=[0.25, 1.35, 0.0, 0.05],
                       sigma=Aerr, absolute_sigma=True)
f_fit, df = popt[1], np.sqrt(pcov[1, 1])
GAMMA = 135.5388                                    # gamma_mu / 2pi in MHz/T
B_fit = f_fit / GAMMA * 1000                        # mT
print(f"events used: {hitU.sum()} (U) + {hitD.sum()} (D)")
print(f"A0 = {popt[0]:.3f} +/- {np.sqrt(pcov[0, 0]):.3f}")
print(f"f  = {f_fit:.4f} +/- {df:.4f} MHz  ->  B = {B_fit:.3f} +/- {df / GAMMA * 1000:.3f} mT (set:
  ↪ 10.000 mT)")

fig, ax = plt.subplots(figsize=(6.5, 4))
ax.errorbar(tg, A, yerr=Aerr, fmt=".", ms=3, label="simulated asymmetry")
tt = np.linspace(tg.min(), tg.max(), 1000)
ax.plot(tt, osc(tt, *popt), "-", label=rf"fit: $f$ = {f_fit:.3f} MHz")
ax.set_xlabel(r"Decay time [$\mu$s]")
ax.set_ylabel(r"Asymmetry $A(t)$")
ax.legend()
fig.tight_layout()
fig.savefig("asymmetry.png", dpi=150)
```

9.4 What you should see

In our reference run, 26 224 positrons were counted in U and 25 956 in D. The fit gives

$$f = 1.3530(20)\,\text{MHz} \quad \Rightarrow \quad B = \frac{f}{\gamma_\mu/2\pi} = 9.983(15)\,\text{mT},$$

within about one standard deviation of the 10.000 mT we set — a full round trip from a number in a macro to a field recovered from simulated positron counts (Figure 9.1). The amplitude, $A_0 = 0.266(9)$, sits below the maximum muon-decay asymmetry of 1/3 (averaged over all positron energies) because each counter accepts positrons over a wide range of angles.

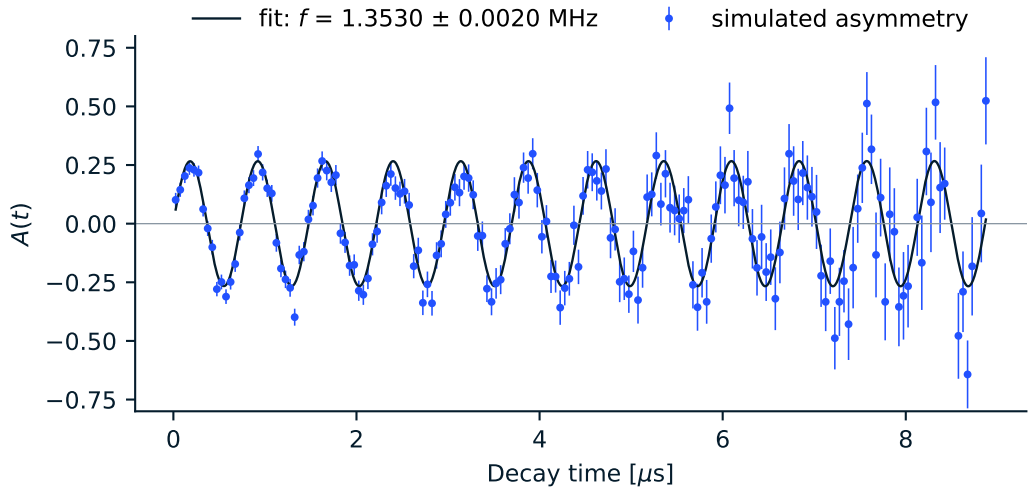


Figure 9.1: Simulated positron asymmetry $A(t)$ from `tf_musr.mac` with the damped-sine fit. Error bars grow at late times as the surviving muons run out.

KEY POINT

This closes the loop of the whole book. Geometry (Chapter 4) placed the counters; the physics block (Chapter 3) let the muons stop and the positrons deposit energy; the field (Chapter 8) drove the precession; the beam and spin (Chapter 5) set the initial condition; the ROOT tree (Chapter 6) recorded `det_ID` and `muDecayTime`; and Python (Chapter 7) turned them into a number with a physical meaning.

TIP

Turn this into a study. Scan the field by editing the one `0.01` in the macro, run each value (give each run its own suffix on the command line), and plot fitted frequency against set field — it should be a straight line of slope $\gamma_\mu/2\pi$. For a mapped field, the `<final> <nSteps>` ramp of Chapter 8 scans the field within a single run. Either is a genuine, if small, piece of μSR physics.

RECAP

- A complete experiment is a sample, two counters, a transverse field, the physics block

and a polarised beam.

- The asymmetry $(N_{\mathrm{D}} - N_{\mathrm{U}})/(N_{\mathrm{D}} + N_{\mathrm{U}})$ cancels the lifetime and exposes the precession.
- Fitting the oscillation recovers the field you set, via $f = (\gamma_\mu/2\pi)B$: $9.983(15)\,\mathrm{mT}$ for $10\,\mathrm{mT}$ set.
- Every skill from Chapters 3–8 appears here as one part of a single physics result.

Next: troubleshooting, and reading the source →

Chapter 10

Troubleshooting, and reading the source

IN THIS CHAPTER YOU WILL

- Diagnose the common build, run-time and analysis failures quickly.
- Turn “unknown keyword” into a ten-second source lookup.
- Know which source file answers which kind of question.
- Build the validation habits that keep simulated numbers trustworthy.

`musrSim` is a forked, evolving codebase, so the most valuable long-term skill is not memorising commands but knowing how to interrogate your own copy. This short chapter is a field guide to doing exactly that.

10.1 Common failures at a glance

Symptom	Most likely cause and fix
Build: `Could NOT find Geant4` / `root-config` missing	Environment not set up. Activate the conda environment, or source `geant4.sh` and `thisroot.sh`, before `cmake`.
Build: `unsuitable version` of EXPAT or another library	CMake mixed system and conda libraries. Configure with `-DCMAKE_PREFIX_PATH=$CONDA_PREFIX`.
Run stops: `Root output file ... can not be created`	No `data/` directory where you launched. `mkdir -p data`.
Run stops: `Unknown keyword requested`	A `/musr/command` keyword your build does not implement, or a typo. Look it up (Section 10.2).
Run stops: `not implemented in musr PhysicsList.cc`	A process name the physics list does not know (Section 8.2).
Run stops: `Logical Volume "..." not found`	Mother written without `log_`, misspelled, or constructed after the daughter.
Run stops: `Field map file not found`	The map is not in `build/run`; put it in `run/` and re-run `cmake ../`.
Tree is empty, or muons never stop	No physics block in the macro (Section 3.5.1), or nothing reaches a sensitive volume.
Geometry or physics lines ignored	They are in a file pulled in with `/control/execute`; `/musr/command` lines must be in the main macro (Section 3.2).
Overlap reported during construction	A daughter exceeds or intersects its mother, or a half-length is wrong (Chapter 4).
Parallel jobs give identical results	Seeding: use `/musr/run/randomOption 1` with a different offset per job (Section 8.4).
Histogram spikes at −1000	Uncut sentinel values (Chapter 6).
Fit returns nonsense	Jagged array flattened before masking, or starved bins at late time (Chapter 7).

10.2 When a command is rejected

For `/musr/command` lines, `musrSim` prints `E R R O R ... Unknown keyword requested` with the offending line and stops. For `/gun`, `/musr/run` and Geant4 commands, Geant4 reports `command not found` and interrupts the batch. Either way the command does not exist in your build — do not fight it; find out what does exist:

Listing 10.1: Confirm a command against your own source

```
# 1. UI commands (/gun, /musr/run, Geant4): ask the running executable
../musrSim first_sim.mac idle      # then: ls /gun/   help /gun/vertexsigma

# 2. /musr/command keywords: grep the parsers
grep -n '"storeOnlyEventsWithHits"' src/musrDetectorConstruction.cc
grep -n '"globalfield"'             src/musrDetectorConstruction.cc
grep -n 'stringProcessName=='       src/musrPhysicsList.cc
```

The grep tells you instantly whether the keyword is implemented, and the `sscanf` a few lines below the match tells you its exact argument order — more reliable than any manual, including

this one.

10.3 A map of the source

You do not need to read the whole codebase. A handful of files answer almost every question.

File	Answers questions about
`musrSim.cc`	Start-up, command-line arguments, run modes, output-name pre-scan, the order of initialisation.
`src/musrDetectorConstruction.cc`	Every `/musr/command` keyword except `process`: solids, rotations, materials, `globalfield`, `rootOutput`, `visattributes`, the magic volume names.
`src/musrPhysicsList.cc`	`/musr/command process ...`: which processes exist and how they are added.
`src/musrRootOutput.cc`	Every ROOT branch: names, types, units and default on/off state; the output file name.
`src/musrPrimaryGeneratorAction.cc`, `...Messenger.cc`	Every `/gun/` command: vertex, momentum, spin, TURTLE, GPS switch-over, EcoMug and CRY.
`src/musrScintSD.cc`	What a hit is and how `det_...` values are computed; hit merging and event selection.
`src/musrSteppingAction.cc`, `musrEventAction.cc`	Per-step and per-event logic: `kill`/`shield` volumes, `save` planes, run-time limits, the `.stop` file.
`src/musrDetectorMessenger.cc`	`/musr/run/...`: random-number options.
`src/F04*.cc`, `musrTabulatedElementField.cc`	Field elements, field maps and integration accuracy.
`include/EcoMug.h`, `src_cry/`	The cosmic-ray generator libraries.

KEY POINT

The pattern to internalise: a `/musr/command X` line is parsed by a long `if/else if (strcmp(...,"X")==0)` chain in `musrDetectorConstruction.cc` (or `musrPhysicsList.cc` for `process`), and a ROOT branch is declared once in `musrRootOutput.cc`. Those files, searched with `grep`, are the definitive reference for this toolkit.

10.4 Validation habits

Simulated numbers are only as good as your scepticism. Four cheap checks catch most mistakes.

1. **Look at the geometry** before any physics run, and confirm every overlap check says `OK`.
2. **Run a known case.** A μ^+ lifetime fit must return 2.197 µs; a field you set must come back out of a precession fit (Chapter 9). If a calibration you understand is wrong, stop.
3. **Check counts and sentinels.** Compare the number of stored events, the `det_n` distribution and the fraction of -1000 values against expectation before trusting a spectrum.
4. **Vary one thing.** Change the field, the momentum or the geometry by a known amount and confirm the result moves the way physics says it should.

RECAP

- Most failures are environment, a missing `data/`, a missing physics block, geometry overlaps, or uncut sentinels.
- A rejected command means grep the source or ask the executable — never guess.
- `musrDetectorConstruction.cc`, `musrPhysicsList.cc` and `musrRootOutput.cc` are the real reference.
- Validate with a known lifetime and a known field before trusting anything new.

Next: reference appendices →

Appendix A

Command quick reference

A summary of every command family, checked against the `musrSim-upgrade` source (`musrDetectorConstruction.cc`, `musrPhysicsList.cc`, `musrPrimaryGeneratorMessenger.cc`, `musrDetectorMessenger.cc`). Units inside `/musr/command` lines are fixed: mm, degrees, tesla, kV/mm, ns. `/gun` commands take an explicit unit.

GEOMETRY

```
/musr/command construct <solid> <name> <dims...> <material> <x y z> <log_mother> <rot> <sens> <
  ↪ volID>
/musr/command construct stl <name> <file.stl> <material> <x y z> <log_mother> <rot> <sens> <volID>
/musr/command rotation <name> <ax ay az angle>        # axis-angle (angle != 0)
/musr/command rotation <name> <phi theta psi>         # Euler angles
/musr/command arrayDef <name> <n> <v1 ... vn>         # arrays for polyconeA/B
/musr/command visattributes <log_name | material> <colour | invisible>
# solids: box tubs cons sphere trd para torus polyconeA polyconeB stl + composites (Appendix C)
# sens:   musr/ScintSD (detector) | dead (inert)
# magic names: World, Target, M0/M1/M2, *save*, kill*, shield*
```

PHYSICS

```
/musr/command process addProcess <particle> <G4Process> <atRest> <alongStep> <postStep>
/musr/command process addDiscreteProcess <particle> <G4Process>
/musr/command process cutForGamma <mm>        /musr/command process cutForElectron <mm>
/musr/command G4OpticalPhotons <true|false>
/musr/command materialPropertiesTable <table> <PROPERTY> <n> <energies...> <values...>   # n=0:
  ↪ constant
/musr/command setMaterialPropertiesTable <table> <material>
/musr/command opticalSurface <name> <phys_vol1> <phys_vol2> <type> <finish> <model> [<table>]
/musr/command killAllElectrons|killAllPositrons|killAllGammas|killAllNeutrinos <true|false>
/musr/command SetUserLimits <log_name> ...      /musr/command region define <region> <log_name>
```

FIELDS

```
/musr/command globalfield <name> <hx hy hz> uniform <x y z> <log_vol> <Bx By Bz> <Ex Ey Ez>
/musr/command globalfield <name> <x y z> fromfile <format> <file> <log_vol> <value> [<final> <
  ↪ nSteps>]
/musr/command globalfield <name> <x y z> quadrupole <halfLength> <radius> <fringe> <log_vol> <
  ↪ gradient>
/musr/command globalfield setparameter <SetLargestAcceptableStep|SetMinimumEpsilonStep|
              SetMaximumEpsilonStep|SetDeltaOneStep|SetDeltaIntersection|SetMaxLoopCount> <value>
/musr/command globalfield printparameters
/musr/command globalfield printFieldValueAtPoint <x y z>
```

OUTPUT AND RUN CONTROL

```
/musr/command SetOutputFileName <DEFAULT | suffix>     # data/musrSim_jp<NNN>[_suffix].root
/musr/command rootOutputDirectoryName <dir>
/musr/command rootOutput <branch-switch> <off | on>
/musr/command storeOnlyEventsWithHits <true|false>
/musr/command storeOnlyEventsWithHitInDetID <id>
/musr/command storeOnlyEventsWithMuonsDecayedInDetID <id>
/musr/command storeOnlyTheFirstTimeHit <true|false>
/musr/command signalSeparationTime <ns>
/musr/command maximumRunTimeAllowed <s>                # default 85000
/musr/command maximumTimePerEvent <s>
/musr/command maximumNrOfStepsPerTrack <n>
/musr/command SetRndSeedOffset <n>     /musr/command SetRndSeed <n>
/musr/run/randomOption <0|1|2|3|4|5>   /musr/run/howOftenToPrintEvent <n>
/musr/run/runID <n>
/run/beamOn <N>
```

BEAM (PARTICLE GUN)

```
/gun/primaryparticle <mu+|mu-|e+|e-|proton|...>
/gun/vertex <x y z> mm             /gun/vertexsigma <sx sy sz> mm    # +Gaussian, -uniform
/gun/vertexboundary <rMax zMin zMax> mm               /gun/vertexrelativer <rMax> mm
/gun/momentum <p> MeV              /gun/kenergy <T> MeV               /gun/momentumsmearing <sigma>
  ↪ MeV
/gun/momentumboundary <pMin pMax 0> MeV
/gun/direction <dx dy dz>          /gun/tilt <ax ay 0> deg            /gun/tiltsigma <sx sy 0> deg
/gun/pitch <angle> deg
/gun/muonPolarizVector <x y z>     /gun/muonPolarizFraction <-1..1>
/gun/decaytimelimits <min max meanlife> ns                  # active only if max > 0
/gun/starttime <t> ns  /gun/starttimesigma <t> ns  /gun/meanarrivaltime <t> ns
/gun/turtlefilename <file>  /gun/turtleZ0position <z> mm  /gun/turtleMomentumScalingFactor <f>
/gun/turtleInterpretAxes <mode>  /gun/turtleMomentumBite <...>  /gun/turtleFirstEventNr <n>
```

COSMIC RAYS (APPENDIX E)

```
/gun/ecomug/useEcoMug true
/gun/ecomug/shapeConstruct <plane|sphere|cylinder> <s1 s2 cx cy cz>          # mm
/gun/ecomug/constraints <pMin pMax thetaMin thetaMax phiMin phiMax>       # MeV/c, deg
/gun/cry/useCRY true
/gun/cry/input <key value>  ...  then  /gun/cry/update       # or: /gun/cry/file <setup file>
/gun/cosmic_v1 <true|false>
```

Appendix B

ROOT output branch reference

The tree is named `t1`. Per-hit arrays all have length `det_n`. Positions are in mm, times in µs, energies in MeV and momenta in MeV/c unless noted; −1000 marks "undefined". Switch off what you do not need with `rootOutput <switch> off`; the switch name differs from the branch name where noted.

EVENT LEVEL (ONE VALUE PER EVENT)

`runID`, `eventID`, `weight`	Identifiers and event weight.
`timeToNextEvent`	For continuous-beam studies (`/gun/meanarrivaltime`).
`BFieldAtDecay`	$B_x, B_y, B_z, E_x, E_y, E_z$ at the muon decay point.
`muIniTime`, `muIniPosX/Y/Z`	Muon start time and position.
`muIniMomX/Y/Z`, `muIniPolX/Y/Z`	Muon initial momentum and polarisation.
`parIniTime`, `parIniPosX/Y/Z`, `parIniMomX/Y/Z`, `parID`	Primary-particle start values (any species; filled by EcoMug and CRY).
`muDecayDetID`, `muDecayTime`	Volume of decay (volumeID) and decay time.
`muDecayPosX/Y/Z`, `muDecayPolX/Y/Z`	Position and spin at decay.
`muTargetTime`, `muTargetPolX/Y/Z`, `muTargetMomX/Y/Z`	On entering a `Target` volume (only when one exists).
`muM0Time`, `muM0PolX/Y/Z` (also `M1`, `M2`)	On entering `M0`–`M2` counters (only when they exist).
`posIniMomX/Y/Z`, `elIniMomX/Y/Z`	Initial momentum of the decay positron / electron.
`nFieldNomVal`, `fieldNomVal`	Nominal values of the defined field elements.
`BxIntegral`, `ByIntegral`, `BzIntegral`, `BzIntegral1..3`	Field path integrals; off by default (switches `fieldIntegralBx`, …).
`nOptPhot`, `nOptPhotDet`, `phot_time`	Optical-photon counts and times (optical runs only).

DETECTOR HITS (ARRAYS OF LENGTH DET_N)

`det_n`	Number of hits this event.
`det_ID`	Detector volumeID of each hit.
`det_edep`	Total energy deposited per hit.
`det_edep_el/pos/gam/mup/mun`	Deposit by e^-, e^+, γ, μ^+, μ^-.
`det_kine_mup/mun`	Kinetic energy of the $\mu^\pm$ at its step in the hit.
`det_time_start`, `det_time_end`	Hit start and end time (switches `det_start`, `det_end`).
`det_x/y/z`, `det_x/y/z_mup`, `det_x/y/z_mun`	First-step position of the hit, and of the $\mu^\pm$ in it.
`det_nsteps`, `det_length`, `det_kine`	Step count, path length, kinetic energy of the first particle.
`det_VrtxKine`, `det_VrtxX/Y/Z`, `det_VrtxVolID`, `det_VrtxProcID`, `det_VrtxTrackID`, `det_VrtxPrtTrackID`, `det_VrtxParticleID`	Provenance of the depositing particle: its creation vertex, volume, process, track, parent track and particle type.
`det_Vvv...` (same set)	The same provenance traced one step further back, to an ancestor of the depositing particle (see `musrScintSD.cc` for the exact rule).

TRACKING PLANES AND OPTICAL DETECTORS (ONLY WHEN USED)

`save_n`, `save_detID`, `save_particleID`	Crossings recorded by `*save*` volumes.
`save_ke`, `save_time`, `save_x/y/z`, `save_px/py/pz`	Kinematics at the plane.
`save_polx/poly/polz`	Spin at the plane.
`odet_n`, `odet_ID`, `odet_nPhot`, `odet_nPhotPrim`	Per optical detector: photon counts.
`odet_timeFirst/Second/Third`, `odet_timeA..D`, `odet_timeMean`, `odet_timeLast`	Photon arrival-time estimators.
`odet_timeCFD`, `odet_amplCFD`, `odet_timeC1..`, `odet_timeCFDarray`	Simulated signal (OPSA): constant-fraction timing and amplitude.

Appendix C

Materials and solids reference

FREQUENTLY USED NIST MATERIALS

`G4_Galactic`	Ideal vacuum.
`G4_AIR`	Air (the usual world).
`G4_PLASTIC_SC_VINYLTOLUENE`	Plastic scintillator.
`G4_Cu`, `G4_Al`, `G4_Pb`, `G4_Fe`, `G4_W`, `G4_Ag`, `G4_Au`	Common metals.
`G4_MYLAR`, `G4_KAPTON`, `G4_POLYETHYLENE`	Films and light materials.
`G4_WATER`, `G4_Si`, `G4_Ge`, `G4_CONCRETE`	Water, semiconductors, concrete.

Any name from Geant4's NIST database works (`G4_...`). In addition, `musrDetectorConstruction::DefineMaterials()` defines: `Scintillator`, `Mylar`, `Brass`, `Steel`, `Water`, `Aerogel`, `LaBr3`, `Macor`, `MCPglass`, `MgO`, `SiO2`, `Al2O3`, `K2O`, `B2O3`, `Air`, `Vacuum`, `ArgonGas`, `Rock`, `Concrete`, low-pressure air (`Air1mbar`, `AirE1mbar` …`AirE6mbar`) and a series of helium gases (`HeliumGas5mbar` …`HeliumGas100mbar`, `HeliumGasSat4K`, …).

SOLIDS AND THEIR DIMENSION FIELDS (MM, DEG; BOX/TUBE EXTENTS ARE HALF-LENGTHS)

`box`	half-x, half-y, half-z
`tubs`	rMin, rMax, half-z, ϕ_0, $\Delta\phi$
`cons`	rMin1, rMax1, rMin2, rMax2, half-z, ϕ_0, $\Delta\phi$
`sphere`	rMin, rMax, ϕ_0, $\Delta\phi$, θ_0, $\Delta\theta$
`trd`	half-x1, half-x2, half-y1, half-y2, half-z
`para`	half-x, half-y, half-z, α, θ, ϕ
`torus`	rMin, rMax, rTorus, ϕ_0, $\Delta\phi$
`polyconeA`	ϕ_0, $\Delta\phi$, n, arrays z, rInner, rOuter (from `arrayDef`)
`polyconeB`	ϕ_0, $\Delta\phi$, n, arrays r, z
`stl`	an STL file name instead of dimensions (CADMesh import, mm)

Composite shapes hard-coded in `musrDetectorConstruction.cc` (read the source for their parameters): `boxbox`, `tubsbox`, `tubsbox2`, `tubsboxsegm`, `cylpart`, `uprofile`, `cruciform`, `trd90y`, `tubeWithWindows`, `TubeWithTubeHole`, `TubeWithHolePlusTubeHole`, `TubeWithHoleAndTubeHole`, `BarWithBevelledEdgesX`, `alcSupportPlate`, and the PSI instrument parts `GPDcollimator`, `GPDmHolder`, `GPDsampleHolderA`, `GPSforward`, `GPSbackward`, `GPSbackwardVeto`.

Appendix D

An annotated skeleton macro

Copy this template and fill in the bracketed parts. It is ordered for readability — geometry, physics, output, beam, go — and keeps every `/musr/command` line in the one file you pass on the command line (Section 3.2).

Listing D.1: A reusable starting point for a new simulation

```
# ================================================================
# <name>.mac -- one-line description
# /musr/command units: mm, deg, tesla, kV/mm, ns
# ================================================================

# ---- rotation matrices (define BEFORE the volumes that use them) ----
# 90 deg about y
/musr/command rotation rotA 0 1 0 90

# ---- geometry (World first; mothers before daughters) ----
/musr/command construct box World <hx> <hy> <hz> G4_AIR 0 0 0 no_logical_volume norot dead -1
/musr/command construct <solid> Target <dims> <material> <x y z> log_World norot dead 5
/musr/command construct box Counter1 <hx hy hz> G4_PLASTIC_SC_VINYLTOLUENE <x y z> log_World norot
  ↪ musr/ScintSD 1
/musr/command construct box Counter2 <hx hy hz> G4_PLASTIC_SC_VINYLTOLUENE <x y z> log_World rotA
  ↪ musr/ScintSD 2

# ---- appearance ----
/musr/command visattributes log_World invisible
/musr/command visattributes log_Target blue

# ---- field (optional): six numbers, B in tesla then E in kV/mm ----
# /musr/command globalfield F <hx hy hz> uniform 0 0 0 log_World <Bx> <By> <Bz> 0 0 0
# /musr/command globalfield printFieldValueAtPoint 0 0 0

# ---- physics (REQUIRED: without it nothing interacts) ----
/musr/command process addDiscreteProcess gamma G4PhotoElectricEffect
/musr/command process addDiscreteProcess gamma G4ComptonScattering
/musr/command process addDiscreteProcess gamma G4GammaConversion
/musr/command process addDiscreteProcess gamma G4RayleighScattering
/musr/command process addProcess  e-  G4eMultipleScattering   -1  1  1
/musr/command process addProcess  e-  G4eIonisation           -1  2  2
/musr/command process addProcess  e-  G4eBremsstrahlung       -1  3  3
/musr/command process addProcess  e+  G4eMultipleScattering   -1  1  1
/musr/command process addProcess  e+  G4eIonisation           -1  2  2
/musr/command process addProcess  e+  G4eBremsstrahlung       -1  3  3
/musr/command process addProcess  e+  G4eplusAnnihilation      0 -1  4
/musr/command process addProcess  mu+ G4MuMultipleScattering  -1  1  1
/musr/command process addProcess  mu+ G4MuIonisation          -1  2  2
/musr/command process addProcess  mu+ G4MuBremsstrahlung      -1  3  3
```

```
/musr/command process addProcess  mu+ G4MuPairProduction      -1  4  4
/musr/command process addProcess  mu- G4MuMultipleScattering  -1  1  1
/musr/command process addProcess  mu- G4MuIonisation          -1  2  2
/musr/command process addProcess  mu- G4MuBremsstrahlung      -1  3  3
/musr/command process addProcess  mu- G4MuPairProduction      -1  4  4

# ---- output ----
/musr/command SetOutputFileName DEFAULT
/musr/command storeOnlyEventsWithHits true
# /musr/command rootOutput odet_ID off
/musr/run/howOftenToPrintEvent 10000
# for parallel production jobs:
# /musr/run/randomOption 1

# ---- beam ----
/gun/primaryparticle mu+
/gun/vertex 0 0 <z> mm
/gun/vertexsigma <sx> <sy> 0 mm
/gun/momentum <p> MeV
/gun/muonPolarizVector 0 0 -1
/gun/muonPolarizFraction 1

# ---- go ----
# /control/execute visVRML.mac
/run/beamOn <N>
```

Appendix E

Cosmic-ray muons: EcoMug and CRY

Muography, shielding studies and detector cosmic tests need primaries that follow the sea-level cosmic-ray distribution rather than a beam. `musrSim` includes two generators for this, plus a simple legacy option. Each replaces the `/gun` beam model for the whole run.

KEY POINT

In both generators $+z$ is *vertical up*: particles start on a horizontal surface and travel downward ($p_z < 0$). Build your geometry with the sky at the top — for example, a building or overburden above a detector at negative z. The generated particle's start values are stored in the `parIni...` branches and its PDG code in `parID` ($\mu^- = 13$, $\mu^+ = -13$); the `muIni...` branches are not filled.

E.1 EcoMug

EcoMug (D. Pagano *et al.*) is a fast, header-only generator of single cosmic muons using a parametrised sea-level spectrum. It is in `include/EcoMug.h`.

Listing E.1: EcoMug on a 5 m × 5 m plane (from `run/EcoMugExample.mac`)

```
/gun/ecomug/useEcoMug true
# shape: plane <sizeX sizeY> <centre x y z>  -- all in mm
/gun/ecomug/shapeConstruct plane 5000 5000 0 0 0
# constraints: pMin pMax [MeV/c]  zenithMin zenithMax [deg]  azimuthMin azimuthMax [deg]
/gun/ecomug/constraints 10 10000 20 60 0 180
```

The generation surface can be a `plane` (`<sizeX sizeY cx cy cz>`), a half-`sphere` (`<radius 0 cx cy cz>`) or a `cylinder` (`<radius height cx cy cz>`); `shapeConstruct` always takes five numbers. The defaults are a 1 m square plane at the origin, full momentum range, zenith 0–90° and azimuth 0–360°. Running the example as shipped produced muons on the $z = 0$ plane within ±2.5 m, all heading downward, with zenith angles between 20° and 60°, and both charges.

CHECK YOUR BUILD

`EcoMugExample.mac` switches off almost every branch, including `parIni...`, to show the generator alone. Delete those `rootOutput ... off` lines when you want to see the generated muons in the tree.

E.2 CRY

CRY (Lawrence Livermore National Laboratory) generates complete cosmic-ray *showers* — muons, neutrons, protons, photons, electrons and pions — with correlated multiplicities, for a chosen date (solar cycle), latitude and altitude. One `musrSim` event is one shower. The library is compiled in from `src_cry/`.

Listing E.2: CRY: muons only, sea level, 5 m sub-box (from `run/CRYExample.mac`)

```
/gun/cry/useCRY true
/gun/cry/input returnMuons 1
/gun/cry/input returnNeutrons 0
/gun/cry/input returnProtons 0
/gun/cry/input returnGammas 0
/gun/cry/input returnElectrons 0
/gun/cry/input returnPions 0
/gun/cry/input date 7-1-2012
/gun/cry/input latitude 90.0
/gun/cry/input altitude 0
/gun/cry/input subboxLength 5
/gun/cry/update
# alternatively, read the same key-value pairs from a file:
# /gun/cry/file CRYSetup.file
```

Each `/gun/cry/input` line adds one key–value pair; `/gun/cry/update` must follow before `/run/beamOn`. `subboxLength` (metres) sets the side of the square area on which CRY places particles; altitude is in metres (CRY supports 0, 2100 and 11300). See the CRY manual for all keys.

> **COMMON PITFALL**
>
> CRY needs its data tables (`cosmics_*.data`), which are not in the repository. Copy the `data/` files of the CRY distribution into `cryData/` at the top of the repository (the default path compiled into `musrSim`), or point to them at run time with `export CRYDATAPATH=/path/to/cry/data`. Without them the run stops with `CRY::CRYData: Error reading .../cosmics_0.data`.

E.3 Legacy option

`/gun/cosmic_v1 true` draws each primary's energy and its angle to the gun axis from a simple built-in cosmic-muon distribution (`cosmicMuonManager`, written by M. Lv), with uniform azimuth, while keeping the `/gun` vertex settings. Unlike EcoMug and CRY it works in the beam frame: the muons travel along $+z$, not downward. EcoMug supersedes it for new work; the examples set it to `false`.

E.4 References

D. Pagano, G. Bonomi, A. Donzella, A. Zenoni, G. Zumerle, N. Zurlo, "EcoMug: an Efficient COsmic MUon Generator for cosmic-ray muon applications," *Nucl. Instrum. Meth. A* **1014** (2021) 165732.

C. Hagmann, D. Lange, D. Wright, "Cosmic-ray shower generator (CRY) for Monte Carlo trans-

port codes," *IEEE Nuclear Science Symposium Conference Record* (2007) 1143–1146.

Appendix F

Environment quick reference

EVERYTHING FROM CONDA-FORGE (TESTED)

```
conda create -n musrsim -c conda-forge geant4 root cmake make cxx-compiler expat
conda activate musrsim
pip install uproot awkward numpy scipy matplotlib
```

CVMFS ON A CLUSTER

```
# one line gives a matching Geant4 + ROOT + compiler (pick your site's LCG release):
source /cvmfs/sft.cern.ch/lcg/views/LCG_101/x86_64-centos7-gcc11-opt/setup.sh
```

BUILD AND RUN

```
git clone https://github.com/kimsiang/musrSim-upgrade.git
cd musrSim-upgrade
mkdir build
cd build
cmake ../            # add -DCMAKE_PREFIX_PATH=$CONDA_PREFIX if CMake mixes in system libraries
make -j4             # -> build/musrSim ; run/ is copied to build/run/
cd run
mkdir data
../musrSim 101.mac   # -> data/musrSim_jp101.root
```

COMMAND LINE

```
../musrSim <macro.mac> [name | idle] [seedOffset]
#   name       -> output suffix: data/musrSim_jp<NNN>_<name>.root
#   idle       -> interactive Geant4 prompt after the macro has run
#   seedOffset -> used by /musr/run/randomOption 1 (independent batch jobs)
```

TUTORIAL FILES IN THE REPOSITORY

`run/first_sim.mac`	Chapter 3: muon beam into one scintillator slab.
`run/tf_musr.mac`	Chapter 9: transverse-field asymmetry experiment.
`doc/scripts/first_analysis.py`	Chapter 7: spectrum and lifetime fit.
`doc/scripts/load.py`	Chapter 7: reusable loader.
`doc/scripts/asymmetry.py`	Chapter 9: asymmetry and Larmor-frequency fit.
`doc/scripts/make_figures.py`	Regenerates the figures in this book from the two runs.

CHECK YOUR BUILD

CRY needs its data tables in `cryData/` (or `CRYDATAPATH`), but only if you use CRY; field-map files are needed only for macros that read them (`14510.mac`, `50121.mac`, the `LEM_*.mac` files): neither is shipped. Everything in this book runs without them.

Appendix G

Glossary

Asymmetry
The normalised difference in positron counts between two opposing detectors, $(N_1-N_2)/(N_1+N_2)$; the primary μSR observable. The muon lifetime cancels in the ratio.

Event
One primary particle (or one CRY shower) tracked with all its secondaries; one stored event is one row of `t1`.

Half-length
Half the full extent of a solid along an axis; the convention Geant4 (and hence `construct`) uses for box and tube dimensions.

Hit
Energy deposited in one sensitive (`musr/ScintSD`) volume within one signal-separation window; an event may contain several, stored as arrays of length `det_n`.

Jagged array
A variable-length-per-event array (like `det_edep`); handled in Python with `awkward`, not plain NumPy.

Larmor frequency
The muon spin-precession frequency in a field, $f = (\gamma_\mu/2\pi)B$ with $\gamma_\mu/2\pi \approx 135.54\,\mathrm{MHz/T}$.

Physics block
The `/musr/command process ...` lines that define a run's electromagnetic physics; without them nothing interacts.

Sensitive detector
A volume that records hits; here `musr/ScintSD`, distinguished per volume by its volumeID.

Sentinel value
A placeholder (-1000, sometimes -999) meaning "not defined for this event"; cut it before analysis.

Steering file
Another name for the macro passed on the command line — the only file whose `/musr/command` lines are read.

Surface muon
A $\approx 29.8\,\mathrm{MeV/c}$, nearly fully spin-polarised μ^+ from pion decay at rest; stops in the first fraction of a millimetre of a sample.

TURTLE

A beam-optics transport code whose output files can drive the `musrSim` beam for realistic beamline studies.

μSR

Muon spin rotation / relaxation / resonance: using the implanted muon's spin as a local magnetic probe of matter.

Appendix H

Further reading

THIS TOOLKIT

- K. Sedlak, R. Scheuermann, T. Shiroka, A. Stoykov, A. R. Raselli, A. Amato, "musrSim and musrSimAna — simulation tools for μSR instruments," *Physics Procedia* **30** (2012) 61–64.
- The PSI μSR simulation pages, `https://www.psi.ch/en/lmu/geant4-simulations`.
- This distribution's source, `https://github.com/kimsiang/musrSim-upgrade` — the definitive reference for your commands and branches.

GEANT4

- S. Agostinelli *et al.*, "Geant4 — a simulation toolkit," *Nucl. Instrum. Meth. A* **506** (2003) 250–303.
- J. Allison *et al.*, "Recent developments in Geant4," *Nucl. Instrum. Meth. A* **835** (2016) 186–225.
- The *Book for Application Developers* and *Physics Reference Manual*, `https://geant4.web.cern.ch`.

COSMIC-RAY GENERATORS

- D. Pagano *et al.*, "EcoMug: an Efficient COsmic MUon Generator for cosmic-ray muon applications," *Nucl. Instrum. Meth. A* **1014** (2021) 165732.
- C. Hagmann, D. Lange, D. Wright, "Cosmic-ray shower generator (CRY) for Monte Carlo transport codes," *IEEE NSS Conf. Record* (2007) 1143–1146.

ANALYSIS TOOLS

- R. Brun, F. Rademakers, "ROOT — an object oriented data analysis framework," *Nucl. Instrum. Meth. A* **389** (1997) 81–86; `https://root.cern`.
- The Scikit-HEP project: `uproot` (`https://uproot.readthedocs.io`) and `awkward` (`https://awkward-array.org`).

μSR PHYSICS

- S. J. Blundell, "Spin-polarized muons in condensed matter physics," *Contemporary Physics* **40** (1999) 175–192.

- A. Yaouanc, P. Dalmas de Réotier, *Muon Spin Rotation, Relaxation, and Resonance* (Oxford University Press, 2011).

END OF TUTORIAL

Every command and branch name in this book was checked against the `musrSim-upgrade` source, and every quoted result was produced by running the shipped macros. When your build disagrees, trust the source — this book has taught you how to read it.